

When Help Hurts: Verification Load and Fatigue with AI Coding Assistants

Guangrui Fan*

School of Computer Science and Technology
Taiyuan University of Science and Technology
Taiyuan, Shanxi Province, China
Faculty of Computer Science & Information Technology
Universiti Malaya
Kuala Lumpur, Malaysia
fgr@tyust.edu.cn

Lihu Pan

School of Computer Science and Technology
Taiyuan University of Science and Technology
Taiyuan, shanxi, China
panlh@tyust.edu.cn

Dandan Liu

Faculty of Arts and Social Sciences
Universiti Malaya
Kuala Lumpur, Malaysia
s2134717@siswa.um.edu.my

Rui Zhang

School of Computer Science and Technology
Taiyuan University of Science and Technology
Taiyuan, ShanXi, China
zhangrui@tyust.edu.cn

Abstract

AI coding assistants help, but developers still spend effort verifying model output. We isolate interface effects by holding a single LLM fixed while $N=60$ participants solve three Python tasks with Inline, Chat, or Structured prompting, plus a no-AI control. AI reduced workload by -18.2 TLX points and time by 22% (25.0 vs. 32.1 min) and improved correctness ($OR=1.71$). Within AI, Inline is fastest and lowest-load on simple work; Chat yields higher correctness beyond a per-observation complexity threshold ($z \approx +0.41$) without a time cost; Structured benefits novices at mid complexity. We introduce a mode-agnostic verification-load index (failures, time-to-first-compile, churn, pauses, switches) that partially mediates rising stress/fatigue across tasks. We translate these findings into design guidance: adaptive mode orchestration, transparency on demand, and verification aware packaging, and propose reporting verification load alongside outcomes to evaluate interfaces as models evolve.

CCS Concepts

• **Human-centered computing** → **Empirical studies in HCI**; **User studies**; • **Software and its engineering** → **Programming teams**.

Keywords

AI-assisted programming; verification load; cognitive load; developer fatigue; trust calibration; human-AI interaction

ACM Reference Format:

Guangrui Fan, Dandan Liu, Lihu Pan, and Rui Zhang. 2026. When Help Hurts: Verification Load and Fatigue with AI Coding Assistants. In *Proceedings of*

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *CHI '26, Barcelona, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2278-3/26/04

<https://doi.org/10.1145/3772318.3791176>

the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26), April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 25 pages. <https://doi.org/10.1145/3772318.3791176>

1 Introduction

AI coding assistants promise faster progress on everyday programming tasks, yet developers still spend real effort *checking and fixing* model output. In practice, the same backend model can feel very different depending on *how* assistance is delivered: inline completions embedded in the editor, chat-style turns, or more structured, form-like elicitation. This paper asks a simple question with practical stakes: when does the *interface* change the balance between help and the burden of verification? Prior studies report meaningful speed gains with assistants alongside mixed correctness and non-trivial validation effort [26, 40, 44, 48]. Common interaction styles span inline completion [45], in-IDE chat and repair [2, 35, 47], and structured prompting to reduce ambiguity [7, 27, 43, 49]. At the same time, fluent but incorrect or insecure suggestions underscore the need for deliberate verification [2, 39, 41].

Despite rapid progress, three gaps limit design guidance for HCI: (i) *Interface vs. model confound*. Many evaluations vary the model or study a single interface, making it hard to attribute effects to interaction design rather than backend capability [16, 49]. (ii) *Process costs are underspecified*. Verification effort is often inferred from outcomes or self-report rather than captured as a portable, *cross-mode* behavioral construct. (iii) *Unclear mode crossovers*. We rarely know *where* along a complexity continuum one interaction style begins to dominate another; coarse bins can mask thresholds [16]. These gaps matter because assistance can add or remove *extraneous load* depending on task structure and user expertise, consistent with Cognitive Load Theory and flow/interruption research [19, 31, 33, 34, 38, 46]. Trust calibration also remains delicate: explanations or confidences do not reliably produce appropriate reliance and can even increase over-reliance [3, 24, 37, 50, 53].

We isolate *interface* effects by holding the LLM backend constant and varying only the interaction mode—*Inline*, *Chat*, and

Structured—in a counterbalanced, within-subjects design (AI cohort) with a matched no-AI control cohort (Figure 1; Section 3). Beyond outcomes, we introduce a *mode-agnostic* behavioral index of **verification-load** that fuses compile/test failures, time-to-first-compile, code churn, pauses, and context switches; higher values mean more checking/repair work. This operationalizes extraneous cognitive load and flow disruption in a form that travels across interaction styles and backends. With the same backend, interface alone materially shifts the assistance–burden trade-off. In our study of $N=60$ developers across three Python tasks, AI assistance (pooled) reduced workload by -18.2 RAW–TLX points and time by 22% relative to no-AI, and improved correctness; *within AI*, *Inline* minimized workload/time on low complexity, *Chat* improved correctness as complexity rose (without time penalties), and *Structured* scaffolding helped novices at mid complexity. Verification-load tracked both subjective burden and correctness, and partially accounted for the increases in stress/fatigue over repeated use. These patterns line up with theory (CLT, flow/interruptions) and with trust-calibration evidence that over- or under-reliance can emerge after “trust shocks” [24, 37, 53].

We organize the study around three questions that connect interface design to outcomes, trajectories, and moderators:

RQ1 Immediate effects. How does interaction mode (*Inline*, *Chat*, *Structured*) affect workload, completion time, and correctness when the LLM backend is held constant?

RQ2 Repeated use. How do stress and fatigue evolve across tasks with and without AI, and does a mode-agnostic verification-load index mediate these trajectories?

RQ3 Moderators and crossovers. How do expertise and continuous task complexity moderate mode effects, and where (along the complexity continuum) do crossovers occur?

This paper makes four interface-level contributions that close the gaps above:

- (1) **Interface attribution under backend parity.** A decoupled, counterbalanced comparison of three modes with a fixed LLM backend, attributing differences to interaction design rather than model power.
- (2) **A portable verification-load composite.** A mode-agnostic, behavioral index that captures process costs (failures, first-compile latency, churn, pauses, switches) and explains when assistance helps or hurts beyond outcomes alone.
- (3) **Complexity thresholds and expertise guidance.** Johnson-Neyman analyses over a continuous complexity measure locate crossovers (e.g., where *Chat* surpasses *Inline* on correctness) and show how these depend on expertise.
- (4) **Actionable design & evaluation recipe.** Interface-level guidance—adaptive mode orchestration, transparency on demand, and verification-aware packaging—plus a standard practice of reporting verification-load alongside outcomes to compare UIs as models evolve.

2 Related Work

2.1 AI Coding Assistants: Modes, Benefits, and Pain Points

AI assistance for programming has evolved from probabilistic auto-completion to IDE-integrated copilots and conversational agents that generate, explain, and repair code. Early transformer-based completion systems demonstrated whole-line/block suggestions embedded in the IDE [45], and subsequent lab/field studies and surveys consistently report meaningful speed gains and reduced rote effort alongside mixed correctness and non-trivial validation effort [26, 40, 44, 48]. Beyond inline completions, chat-based and structured prompting workflows aim to elicit intent, explain behavior, and scaffold testing/repair [2, 7, 27, 35, 43, 47].

Recent HCI/SE work has started to unpack *how* developers actually use these systems. An in-depth field/lab study of developers interacting with code-generating models documents the rhythms of accepting, editing, and rejecting suggestions, and the circumstances under which developers pivot to conversation or manual coding [4]. Complementing this, empirical analyses of IDE autocomplete find that perceived benefit often comes less from keystroke savings and more from *information* benefits (e.g., surfacing API syntax or parameters in context), while also highlighting costs that can shift verification effort back to the user [16]. Interface-level levers are therefore salient: uncertainty highlighting and rationale presentation can alter reliance on completions, but heavy-handed cues risk adding overhead [49]. Moreover, studies of analyst/coding workflows show that the *timing and type* of assistance (planning vs. execution) shape perceived utility and trust, suggesting opportunities for adaptive orchestration between *Inline*, *Chat*, and more structured scaffolds [12, 55].

Gap. Much of literatures evaluates a single mode or changes the model while changing the interface, making it difficult to attribute outcomes to interaction design rather than model capability. *Our contribution* is to hold the LLM backend constant and compare three commonly deployed interaction modes (*Inline*, *Chat*, *Structured*), allowing us to attribute observed differences in workload, time, and correctness to the *interface* rather than the model, and to derive mode-specific guidance conditioned on task complexity and expertise.

2.2 Verification Burden and Security Risk

While assistants can accelerate routine edits, verification remains a central—and costly—part of developer work. Empirical and audit studies show that assistants sometimes produce fluent but incorrect or insecure code; developers then *must* check, adapt, and test suggestions [2, 48]. Security analyses report that AI-suggested code may include vulnerable patterns; users may also exhibit overconfidence in assisted solutions [39, 41]. In the wild, scans of GitHub projects reveal security weaknesses correlated with assistant-generated contributions, underscoring that the burden of verification is not merely an accuracy problem but a reliability and security problem [10]. Replication and repair studies further catalog recurring vulnerability classes and propose remediation flows, pointing to design space for assistant-supported verification and secure-by-default suggestions [30].

These findings align with reports that developers often accept plausible snippets and then incur rework through compile/test loops, edits, and back-and-forth clarification [4, 48]. Yet most evaluations operationalize verification indirectly (e.g., outcomes or self-report) rather than as a *behavioral, cross-mode* construct.

Gap. We lack a mode-agnostic index that captures the *process costs* of checking AI output across Inline, Chat, and Structured interfaces and that ties those costs to downstream well-being. *Our contribution* is a verification-load composite (compile/test failures, time-to-first-compile, churn, pauses, switches) that is portable across modes and statistically mediates repeated-use stress/fatigue trajectories, supporting design implications such as verification-aware packaging, transparency-on-demand, and security co-review.

2.3 Cognitive Load, Flow, and Interruption in Software Work

Cognitive Load Theory distinguishes intrinsic load (task complexity), extraneous load (avoidable costs due to presentation/interface), and germane load (productive schema building), and predicts expertise reversal effects [8, 19, 36, 46]. In programming, code comprehension engages language and executive systems and exhibits expertise effects, implying that the same assistance may reduce or increase effort depending on *task complexity* and *user expertise*. In parallel, HCI and SE research show that interruptions and context switching increase stress, raise resumption costs, and degrade flow [31, 33, 34, 38]. These theoretical and empirical lenses suggest concrete behavioral proxies for extraneous load and flow disruption—e.g., compile/test failures, delayed time-to-first-compile, code churn, pauses, and editor/test/chat switches—that should covary with subjective workload and repeated-use fatigue.

Gap. Although CLT and flow/interruptions have been extensively studied, they are rarely *operationalized* in AI-assisted programming as a portable, *mode-agnostic* behavioral composite and used to explain when assistance helps or hurts across interfaces. *Our contribution* is to instantiate such a composite (verification-load), show that it varies systematically by interaction mode and complexity (with expertise moderation), and demonstrate that it partially mediates increases in stress and fatigue across tasks—thereby linking theory to actionable interface design (e.g., adaptive mode switching and on-demand transparency) in a way that is independent of model capability.

2.4 Trust calibration and uncertainty cues in AI-assisted programming

Appropriate reliance on automation depends on *calibrated trust*: over-reliance induces automation bias and uncritical acceptance, whereas under-reliance leaves assistance value unrealized [11, 24, 37]. Across HAI settings, explanations and confidence displays do not uniformly improve calibration; their effects are task- and user-dependent and can even increase reliance without improving outcomes [3, 29, 50, 53, 54]. In programming specifically, assistants can produce fluent but incorrect or insecure code that *must* be verified before adoption, creating a practical need for interfaces that support appropriate skepticism at the right moments [2, 39, 41, 47].

Recent work begins to specify trust and uncertainty for code assistance at the UI level. Developer studies of code generation tools identify trust needs around controllability, error visibility, and provenance, and argue for design interventions that scaffold appropriate reliance rather than generic confidence readouts [26, 51]. In code completion, generation probabilities alone are poor proxies for the *actionable* uncertainty developers need; UI-side *uncertainty highlighting* can improve error catching and reduce unproductive acceptance without heavy cognitive overhead [49]. Complementing these code-centric results, experiments with language-model uncertainty phrasing show that brief hedged expressions can reduce inappropriate compliance more effectively than numeric confidences, though benefits are context-dependent and can trade off with satisfaction [20]. More broadly, confidence displays can shift users' own confidence and risk miscalibration if presented without care [25]. Qualitative studies of in-IDE assistance further document patterns—accept, edit, reject cycles; rejection streaks; context switching—that foreshadow the behavioral traces of over- or under-reliance [16].

Gap. Most trust/uncertainty papers study *one* interaction style (e.g., Inline OR Chat) or vary cues while simultaneously changing the model, making it difficult to attribute effects to *interface* versus *backend*. Moreover, few works quantify the *behavioral cost of verification* (rework loops, time-to-first-compile, pause/switch patterns) that links reliance dynamics to developer burden across modes. We hold the model fixed and compare *Inline*, *Chat*, and *Structured* interfaces head-to-head, treating reliance as an *interaction-design* problem rather than a model property. We operationalize a *mode-agnostic, behavioral* index of verification work (compile/test failures, time-to-first-compile, code churn, pauses, context switches) and use it to explain where uncertainty/transparency are best surfaced *on demand*, consistent with code-specific evidence favoring targeted highlighting over always-on confidence [20, 25, 49, 51].

2.5 Methods to isolate interface effects and analyze moderators

Attributing outcomes to *interfaces* rather than *models* requires designs that vary presentation while holding predictions constant. Although some HAI studies manipulate explanations or uncertainty with a fixed predictor [3], many code-assistant evaluations still conflate UI differences with stronger backends, obscuring interface effects [49]. From an experimental-design perspective, counter-balanced within-subjects schedules (e.g., Williams designs) mitigate order and carryover when comparing multiple conditions in the same participants [52]. For analysis, linear mixed-effects models (LMMs) handle participant and task heterogeneity; standard practice uses *lme4* for estimation and *lmerTest* for Satterthwaite/Kenward–Roger inference, with random-effects structures justified by design and simplified when convergence or power considerations warrant [5, 21, 28, 32].

Because key moderators in HCI/SE are often continuous (e.g., per-observation complexity, experience), arbitrary binning should be avoided. The Johnson–Neyman (J–N) technique provides principled *regions of significance* for interactions in regression and multi-level models, clarifying *where* one interface overtakes another along

a continuum [6, 9, 17, 42]. When analysts intend to claim “no practically important difference” between conditions (e.g., two modes are equivalent on workload within a small margin), equivalence testing complements null-hypothesis tests by bounding effects within a *smallest effect size of interest* [22, 23]. Mediation analyses further support process claims when a theoretically motivated mediator is available [14].

Gap. Many prior comparisons neither enforce *backend parity* nor probe *continuous* moderators with J–N, making it difficult to ascribe differences to the UI or to specify *where* a mode becomes preferable. We adopt a decoupled, counterbalanced Task×Mode schedule under *backend parity*; analyze outcomes with LMMs; report *Johnson–Neyman* regions over continuous per-observation complexity to locate crossovers; and use equivalence tests to bound practically small gaps. We pair these methods with a portable verification-load composite to standardize UI evaluation independent of model power [5, 6, 14, 21–23, 28, 32, 42, 52].

2.6 Synthesis and positioning

The literature on AI-assisted programming documents meaningful speed gains alongside mixed correctness, verification frictions, and reliance challenges, yet most comparisons conflate model capability with interface design and seldom quantify the process costs of checking AI output across modes. Building on this background, our work addresses four concrete gaps with interface-focused contributions:

- **UI vs. model confound.** Prior evaluations often vary the model when comparing interaction styles, limiting interface-level attribution. *We hold the LLM backend fixed and vary only the interaction mode (Inline, Chat, Structured), enabling causal attribution to interface design.*
- **No cross-mode verification metric.** Existing studies rarely provide a mode-agnostic, behavioral index of the costs developers incur to check AI output. *We introduce a portable verification-load composite (compile/test failures, time-to-first-compile, code churn, pauses, context switches) with acceptable reliability to standardize UI evaluation independent of model power.*
- **Unclear where modes cross over.** Evidence on when a mode becomes preferable typically relies on coarse task bins or anecdote. *We apply Johnson–Neyman analysis over continuous task complexity to identify complexity-dependent crossover regions and provide expertise-specific guidance on mode selection.*
- **Limited practical guidance.** Design implications are often stated qualitatively and without process-sensitive triggers. *We translate findings into actionable rules for adaptive mode orchestration, transparency-on-demand (rather than always-on), and verification-aware packaging, alongside a standard evaluation recipe that reports outcomes together with verification-load.*

These gaps motivate our mixed-methods, counterbalanced evaluation under backend parity and the construction of a portable verification-load index, as detailed next in Section 3.

3 Method

Our methods isolate *interaction design* from *model capability* to answer three questions: (RQ1) the immediate effects of mode on workload, time, and correctness under backend parity; (RQ2) whether repeated-use trajectories in stress/fatigue differ with and without AI and whether a mode-agnostic *verification-load* mediates these trajectories; and (RQ3) how expertise and continuous task complexity moderate mode effects. Each design choice below (counterbalanced within-subjects AI cohort, between-subjects no-AI cohort, fixed decoding, and unified logging) serves one or more RQs while controlling confounds (Section 2.5).

3.1 Study Overview

We conducted a comparative, mixed-methods study to examine how three AI interaction modes for programming—Inline Suggestions, Chat-based Interaction, and Structured Prompts—shape developers’ cognitive load, performance, stress, and fatigue across multiple tasks. The design comprises:

- D1. Within-subjects (AI cohort).** Each participant in the AI cohort completed three programming tasks, one per interaction mode, in a counterbalanced order (Section 3.3). This enables sensitive, participant-controlled contrasts among the three modes.
- D2. Multi-task no-AI control (between-subjects).** A separate cohort completed the same three tasks without AI assistance, in counterbalanced task order. This establishes concurrent trajectories for repeated use without AI and avoids conflating AI effects with generic multi-task fatigue.
- D3. Exploratory next-day follow-up.** A subset of participants returned on the next day to repeat a matched-complexity task in their assigned condition (AI or no-AI). Results were exploratory and did not change primary conclusions; see Section 5.5 and Appendix Table 22.

Figure 1 overviews assignment, task flow, and the optional next-day follow-up. To avoid confounds from task–mode pairing and task order, we used a decoupled Task×Mode schedule that balances all nine pairings and mode positions equally in the AI cohort (Table 3); the Control cohort used fully balanced task orders. All sessions were screen-recorded (with consent) and instrumentation logs were collected from the IDE and test harnesses to derive behavioral and verification-load indicators. Subjective measures were administered pre-/post-task.¹

3.2 Participants

We recruited $N = 60$ participants comprising professional software engineers and advanced computer science students via departmental mailing lists, professional forums, and local developer communities. Inclusion criteria required: (i) ≥ 2 years of programming experience, (ii) proficiency in Python (self-rated $\geq$ intermediate), and (iii) familiarity with at least one AI-based coding assistant (e.g., ChatGPT, GitHub Copilot, Cursor). Respondents failing to meet criteria were thanked and excluded.

Participants were assigned to (a) an AI cohort ($n = 36$) or (b) a no-AI control cohort ($n = 24$) via blocked randomization stratified by experience bin (Table 1) and per-tool AI familiarity

¹Details of instruments and logging appear in Section 3.5 and Appendix.

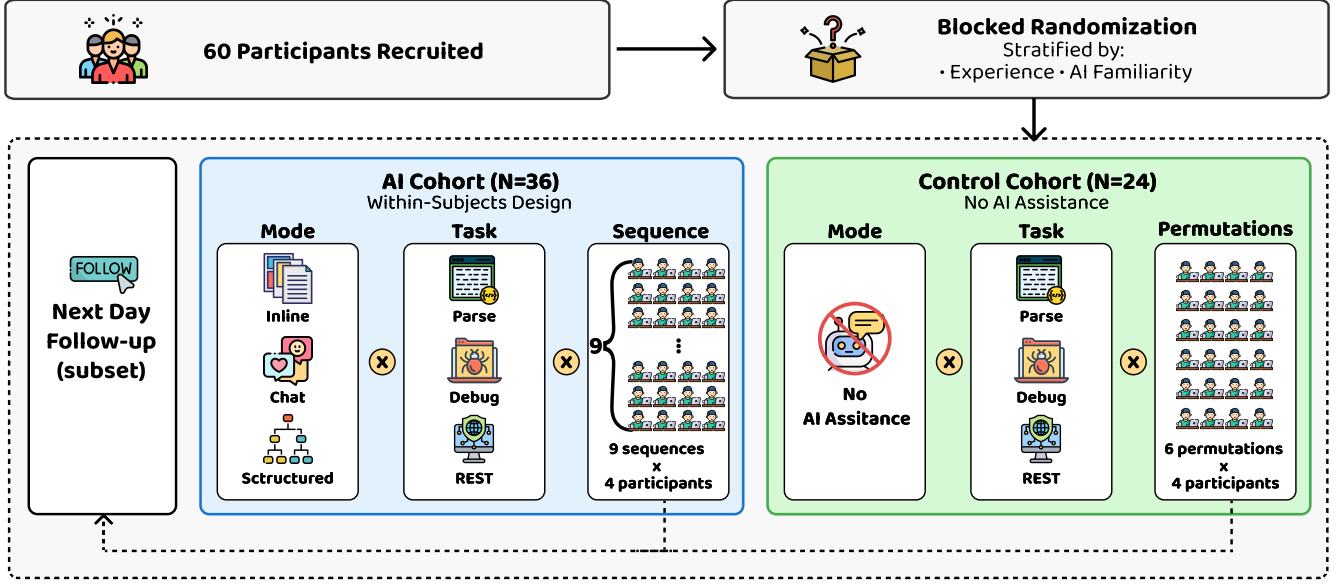


Figure 1: Study flow, counterbalancing, and measurement timeline. $N=60$ randomized to AI ($n=36$; within-subjects comparison of Inline, Chat, Structured across 9 balanced Task $\times$ Mode sequences) or Control ($n=24$; no-AI; balanced task orders). Each task block: pre-task KSS/SSSQ, coding with unified IDE/build logging (events feed the verification-load composite), post-task RAW-TLX/KSS/SSSQ; 40 min cap; 15–20 min breaks; optional next-day follow-up. Only the UI differs; the backend and test oracles are the same. Backend parity: GPT-4o (OpenAI), fixed decoding, no tools/web, state reset.

Table 1: Experience bin rubric used for stratification and moderator analysis. Self-efficacy and AI familiarity were measured on 5-point Likert scales (1=low, 5=high).

Bin	Years coding	Self-efficacy	AI familiarity (per-tool)
Novice	2–3	2–3	1–2 (rarely/never)
Intermediate	3–6	3–4	2–4 (monthly/weekly)
Experienced	≥ 6	4–5	3–5 (weekly/daily)

(weekly/daily vs. monthly/rarely). This procedure yielded baseline equivalence across cohorts on age, years coding, baseline KSS/SSSQ, and AI familiarity (Section 5.1). To make experience operational and reproducible, we used an explicit binning rubric combining years of programming, self-efficacy ratings, and AI-tool familiarity (Table 1). Per-tool familiarity was captured via frequency-of-use items (Daily/Weekly/Monthly/Rarely/Never) and prior task types performed with each tool (e.g., code generation, debugging, test authoring). We collected age, gender, education, primary languages, and domain specialization (e.g., backend, data, web). We also recorded prior exposure to CSV/JSON parsing, auth/debugging, and REST/batch-processing to control for task-specific familiarity.

Power and sample size. We targeted sensitivity to small-to-medium within-subjects mode effects while preserving a concurrent no-AI trajectory. A preregistered simulation using pilot intraclass correlations indicated $\approx 80\%$ power to detect small-to-medium main

effects of Mode on RAW-TLX and log-time within the AI cohort and medium effects for AI vs. Control comparisons at $\alpha=0.05$ (Holm within contrast families). Details appear in Appendix H.8.

3.3 Tasks & Materials

We selected three realistic programming tasks (Table 2) in Python to reflect common professional activities while controlling for language-specific confounds. Each task included starter code, unit tests, and a succinct brief describing functional and non-functional requirements. All code and tests will be made available in anonymized supplemental materials. Tasks A-C cover common professional activities (data parsing, security-relevant debugging, networked batch I/O) with controlled oracles and bounded scope, enabling comparable completion in a single session while eliciting distinct sources of intrinsic complexity (branching, state/clock edge cases, I/O/backoff).

We standardized on Python 3.11 with pytest for unit testing, pytest-cov for coverage, and mutmut for mutation testing. For the REST task we used httpx with a deterministic mock server to control response timing and failure modes. Starter projects included a minimal pyproject.toml and requirements.txt. Each brief specified: (i) input/output contracts (types and shapes), (ii) constraints (e.g., no network I/O for Task A, constant-time token checks for Task B, bounded concurrency for Task C), (iii) interface signatures to implement, and (iv) acceptance criteria expressed as unit tests. Participants were instructed to “make tests pass” and keep changes focused; large refactors were discouraged unless necessary for correctness.

To triangulate task difficulty and avoid advantaging any mode:

Table 2: Task overview, inputs/outputs, and adequacy of test suites. Static metrics were computed on starter code plus scaffolding; test adequacy summaries are computed on the final reference implementations.

Task	Brief	I/O and APIs	Static metrics	Coverage	Mutation
A: Parse	Implement a robust parser that extracts summary statistics (counts, numeric aggregates) from semi-structured CSV/JSON with missing fields and type noise. Enforce schema validation and produce a typed summary object.	Files: CSV/JSON; Output: Python dataclass; No external APIs.	LOC ≈ 200 ; median cyclomatic $\tilde{v} \in [3, 5]$; fan-in/out low.	Line $\geq 80\%$; Branch $\geq 75\%$ (pytest-cov).	Mutation (mut-mut) $\geq 60\%$.
B: Debug Auth	Diagnose and fix logic errors in a minimal authentication module (password hashing, token issuance, expiry). Improve tests to cover edge cases (clock skew, invalid tokens).	In-memory store; hashlib/secrets; No network.	LOC ≈ 260 ; median cyclomatic $\tilde{v} \in [4, 6]$; moderate branching.	Line $\geq 80\%$; Branch $\geq 75\%$.	Mutation $\geq 60\%$.
C: Batch REST	Extend a REST client to perform batched GET requests with rate limiting, exponential backoff, partial failure handling, and JSON aggregation; log and return per-item status.	HTTP via httpx; JSON I/O; Local log.	LOC ≈ 300 ; median cyclomatic $\tilde{v} \in [5, 7]$; I/O branching.	Line $\geq 80\%$; Branch $\geq 75\%$.	Mutation $\geq 60\%$.

- **Expert ratings.** Two senior engineers independently rated perceived complexity (1–5) along structure, branching, and integration dimensions. Inter-rater agreement was substantial ($\kappa \geq 0.70$); disagreements were resolved by discussion.
- **Static metrics.** We computed LOC, cyclomatic complexity (per function and median), and dependency fan-in/out for the starter code. Tasks were targeted to low-to-moderate (A), moderate (B), and moderate-to-high (C) bands without extreme outliers.
- **Pilot equivalence.** A 6-person pilot verified that tasks fell within the intended 20–35 minute window for proficient developers and produced comparable perceived difficulty (no significant differences at $\alpha = 0.05$ on a Raw-TLX composite). Briefs and tests were refined to remove ambiguity uncovered in the pilot.

For primary models we use PreTaskComplexity, defined as the task-level median cyclomatic complexity computed on the starter code (per task), scaled to z across all observations; this avoids post-treatment bias inherent to per-observation edits. Correctness was operationalized as the number of passing unit tests out of a predefined suite per task (10–12 tests including edge cases). We report

line and branch coverage thresholds and mutation scores on the reference implementations (Table 2); participant solutions were run against the same suites. Mutation analysis (“kill rate”) served as a sanity check that tests were sensitive to common faults. For Task C, the mock server produced deterministic error sequences to ensure reproducible backoff behavior.

In addition to task-level static metrics, we computed a per-observation complexity score as the median cyclomatic complexity of functions modified by the participant in that task (weighted by lines changed). This continuous metric (z -scored across all observations) is used as Complexity in LMMs; task identity remains in the model to capture non-metric differences. Because the set of functions modified may itself be influenced by Mode, per-observation complexity is treated as an exploratory moderator in RQ3 rather than a covariate in primary RQ1 models to avoid post-treatment bias. Primary models adjust for pre-task complexity using task-level starter-code metrics. While Task B invites participants to add or refine visible tests as part of the debugging activity, correctness for analysis used a hidden, immutable evaluation suite that was never exposed in the IDE. Participant-authored tests did not affect scoring. To prevent memorization and to diversify starting points, each task had two minor starter variants differing in non-functional details (e.g., identifier names, small refactorings); we denote this as TaskVersion and include it as a random effect in all models.

Counterbalancing and Assignment. To avoid confounding Task with Mode, we balanced all $3 \times 3 = 9$ Task×Mode pairings. We constructed three perfect matchings between Tasks {A,B,C} and Modes {Inline, Chat, Structured} and rotated task order to balance position (Index 1–3). Table 3 lists the nine sequence templates; each was instantiated four times (36 AI participants), so each Task×Mode pairing occurred exactly 12 times overall (and 4 times at each Index). Control participants followed balanced task orders with AI disabled. Assignment to sequence templates was randomized with stratification by experience and AI familiarity and was checked with Fisher’s exact tests (no associations; see Section 5.1).

Allocation details. Sequence templates (Table 3) were generated before recruitment and assigned via blocked randomization stratified by ExperienceBin and AI familiarity. Allocation was concealed until session scheduling. Balance checks (Fisher’s exact tests) found no association between sequence and experience/familiarity (Section 5.1).

3.4 Interaction Modes

We operationalized three AI interaction modes that are commonly available in professional development environments and controlled the underlying model backend so that differences are attributable to interaction style rather than model capability. Figure 7 illustrates the three interfaces; only the UI differed, the LLM backend was identical.

Inline Suggestions (IDE completions). Participants used an IDE with inline code completions enabled (e.g., GitHub Copilot–style). Suggestions appeared as ghost text during typing or on demand via a shortcut. Participants could accept, partially accept, or reject suggestions. We instrumented the IDE to log suggestion

Table 3: Decoupled Task×Mode schedule (AI cohort). Each row shows the (Task, Mode) pair at Index 1, 2, 3. Across the 9 sequences, all 9 Task×Mode pairings are equally represented.

Seq.	Index 1	Index 2	Index 3
S1	(A, Inline)	(B, Chat)	(C, Structured)
S2	(B, Chat)	(C, Structured)	(A, Inline)
S3	(C, Structured)	(A, Inline)	(B, Chat)
S4	(A, Chat)	(B, Structured)	(C, Inline)
S5	(B, Structured)	(C, Inline)	(A, Chat)
S6	(C, Inline)	(A, Chat)	(B, Structured)
S7	(A, Structured)	(B, Inline)	(C, Chat)
S8	(B, Inline)	(C, Chat)	(A, Structured)
S9	(C, Chat)	(A, Structured)	(B, Inline)

events (offered, accepted, edited-after-accept, rejected) with timestamps and cursor locations.

Chat-based Interaction (prompt/response). Participants interacted with the same LLM backend through a dedicated chat panel (dockable in the IDE). They posed natural-language prompts (e.g., “Explain why these tests fail,” “Generate a batch request loop with retries”), received responses, and could copy/paste code into the editor. We logged prompt turns, response lengths, code-block extractions, and paste events.

Structured Prompts (form schema). Participants completed a structured form that constrained the request (schema below): function signature, pre/post-conditions, constraints (e.g., complexity, I/O), error handling, and test hints (edge cases). Submissions triggered LLM code generation and a one-paragraph rationale. We logged field values and generation events.

Structured prompt schema (excerpt)

```
language, function_signature, preconditions[],
postconditions[], constraints[], error_handling[],
test_hints[], style_guidelines[], explanation-
required: bool
```

Permitted affordances per mode. Inline: participants could accept full or partial suggestions and immediately edit accepted code. Chat: participants could request explanations or code, copy/paste code blocks, and ask follow-ups; no external tools were available. Structured: participants completed required fields (signature, pre/post-conditions, constraints, error handling); the form produced code plus a brief rationale.

Unified logging hooks. Representative UI snapshots of the three modes appear in Appendix Figure 7.

Unified logging hooks. All modes wrote into the same event schema (Appendix Table 14); Inline logged offer/accept/reject/edit; Chat logged prompt/response turns and paste events; Structured logged field values and generation events. Build/test outcomes were captured uniformly across modes.

AI Backend and Configuration. Backend parity. All modes called the same LLM backend (GPT-4o, OpenAI, version 2024-11-20) with identical decoding (fixed temperature/top- p , max tokens), no tool or web access, and state reset between tasks. Inline suggestions were gated to syntactically stable points; Chat used a fixed system prompt; Structured serialized fields into a canonical prompt

template. Full parameters and prompt templates are provided in Appendix B.

3.5 Measures

The main text focuses on why each measure is used, what it reveals for our research questions (RQs), and how to interpret; full instruments, scoring, reliability, event-log fields, and construction details (including PCA checks) appear in Appendix E and Appendix H.

Workload (RAW NASA-TLX [13]). We measured perceived workload with the post-task RAW NASA-TLX, a 0–100 composite averaging Mental Demand, Temporal Demand, Effort, and Frustration. This captures perceived burden beyond speed and correctness and is sensitive to interface-induced *extraneous* load (RQ1). Lower scores indicate lower workload; differences of roughly 5–10 points are typically noticeable. As a subjective, single-task measure, it is interpreted alongside repeated-use KSS/SSSQ and behavioral traces.

Completion time (minutes). Completion time is defined as minutes from the first keystroke to the first fully passing test suite. It serves as a direct productivity proxy under identical oracles (RQ1); lower values indicate faster completion. We model time on the log scale to handle skew and report back-transformed means and time ratios (e.g., a ratio of 0.78 corresponds to 22% faster), with right-censoring at 40 min addressed via log models and AFT sensitivity checks.

Correctness. Correctness is the number of unit tests passed out of a task-specific suite ($N=10-12$), providing a comparable outcome-quality indicator across modes (RQ1). Higher values indicate better solutions, and between-mode differences are summarized using odds ratios from GLMMs. Because test suites are proxies for true correctness, mutation and coverage analyses in Appendix C support their adequacy.

Fatigue (Karolinska Sleepiness Scale, KSS [18]). Fatigue is measured with the Karolinska Sleepiness Scale (1 = alert, 9 = very sleepy) administered pre- and post-task. This tracks cumulative tiredness across repeated tasks where TLX may not (RQ2); lower scores indicate more alertness, and we model both post-task levels and pre-post changes (Δ). As a single-item measure, KSS is interpreted in conjunction with SSSQ and behavioral mediators.

Stress (Short Stress State Questionnaire, SSSQ [15]). Stress is assessed with the Short Stress State Questionnaire, computed as a post-task composite and standardized to z-scores. This captures distress and worry beyond effort (RQ2); lower z indicates lower stress. As a self-report measure, SSSQ patterns are triangulated with behavioral traces.

Verification-load composite V_{core} (mode-agnostic). Verification load V_{core} is the mean z-score across five uniformly logged signals: (i) compile/test failures, (ii) time-to-first-compile, (iii) normalized code churn, (iv) cumulative pauses >2 s, and (v) context switches (editor $\leftrightarrow$ tests $\leftrightarrow$ chat/form). This composite operationalizes the *process cost of checking and repairing* model output across interfaces, aligning with cognitive-load and flow/interruptions theory (RQ2/RQ3). Values are unitless within-task z-scores, where 0 is typical

and higher values indicate greater verification burden; an equal-weight scheme is used for portability, with PCA-weighted and mode-extended variants (V_{plus}) reported in Appendix H.

Mode-specific signals (secondary). We also log mode-specific traces, including inline acceptance/rejection densities and Chat prompt turns/pastes, as diagnostic context for mode differences and sensitivity checks. Calibrated acceptance (neither near-0 nor near-1) is descriptively associated with higher correctness, but these signals are not directly comparable across modes and are therefore excluded from V_{core} .

Reliability (e.g., TLX α , SSSQ α), logging coverage ($> 99\%$ for key events), missingness ($< 2\%$), and imputation checks are summarized in the Results and detailed in Appendix E and Appendix H.

3.6 Procedure

Participants reviewed consent, demographics, and expertise/familiarity items, then completed a 5–7 min warm-up with IDE, test runner, and (AI cohort) all three interaction modes. For each task, we administered KSS and SSSQ pre and post. Each participant completed three tasks separated by 15–20 min breaks. The task schedule followed the balanced templates in Table 3 (AI) or a balanced task order (Control). A 40 min cap per task was enforced. If no full pass was achieved by the cap, time was right-censored at 40 min and a completion indicator was recorded; survival (AFT) models serve as robustness checks for time outcomes.

A subset returned the next day to repeat a matched-complexity task in their assigned condition. We re-administered SSSQ/KSS and logged the same behavioral/performance measures to assess short-term recovery and persistence of preferences. Lab sessions used identical workstations; remote sessions mirrored the setup with a provided container image. All sessions were screen-recorded; only derived action codes (navigation, acceptance/rejection, re-runs, context switches) are released. Personally identifying information never appeared in logs. Appendix Figure 8 summarizes the within-task measurement timeline and enforced breaks.

Environment parity. Lab and remote sessions used the same containerized setup; the Environment factor was included in models and did not alter inferences (Section 5.1).

3.7 Instrumentation & Logging Reliability

We synchronized IDE and test-harness clocks via a handshake at task start; median drift was < 50 ms and corrected during alignment. Event coverage exceeded 99% for offers/accepts/rejects, prompts, compile/test runs. Missingness was $< 2\%$ overall and treated as MCAR (Appendix F); primary analyses used listwise deletion with multiple imputation as a sensitivity check (robust).

4 Data Analysis

We map analysis choices directly to the RQs and report effects in units that are easy to read (points, minutes/ratios, odds ratios). Full model syntax, diagnostics, robustness, and equivalence tests appear in Appendix G and H.

For **RQ1**, we compare Inline, Chat, and Structured on workload, time, and correctness using mixed-effects models that adjust for task, order, and experience. Workload (TLX) and time use linear

mixed-effects models (LMMs) with random intercepts for participants and task variant; time is modeled on the log scale but reported back-transformed in minutes alongside time ratios. Correctness uses a binomial GLMM (logit link) over tests passed out of N . These choices isolate interface effects while accounting for repeated measures and task heterogeneity.

For **RQ2**, we estimate fatigue (KSS) and stress (SSSQ) slopes across Task Index (1→3) with LMMs; when supported, we include random slopes for Index by participant to capture individual trajectories. To examine mechanism, we use a two-equation mediation: Index (or Mode) predicts the verification-load composite V_{core} , and V_{core} predicts outcomes while controlling for Index/Mode and covariates. Indirect effects are estimated via Monte-Carlo simulation with bootstrap sensitivity; we interpret significant indirect paths as *consistent with* a verification-load mechanism rather than causal proof.

For **RQ3**, we include interactions with expertise bins and with continuous per-observation complexity. We then use Johnson-Neyman regions to state where, along the complexity continuum, one mode reliably overtakes another. This avoids arbitrary binning and yields actionable thresholds.

Across models we include, unless noted, fixed effects for Mode (or Condition: AI vs Control), Task, Task Index (1–3), Experience-Bin, AI_Familiarity, PreTaskComplexity (task-level), and Environment (lab/remote), plus random intercepts for Participant and TaskVersion; when convergence and fit support it, we add random slopes for Index. We report estimated marginal means with 95% CIs, Holm-adjusted p -values within contrast families, and effect sizes in practical units: TLX point differences (negative is better), back-transformed minutes and time ratios (e.g., 0.78=22% faster), and odds ratios for correctness (> 1 favors the named mode). For KSS/SSSQ we report post-task levels and slopes (lower is better; SSSQ as z). Multiplicity is controlled via Holm adjustments; where we claim “no practically important difference,” we use TOST equivalence with preregistered bounds (± 5 TLX points, $\pm 5\%$ time ratio, $\pm 0.5/10$ tests). Robustness checks (AFT survival for time to handle right-censoring, ordinal models for TLX, alternative links for correctness), influence diagnostics, and multiple-imputation results are in Appendix H. The main analytical limitations are that V_{core} is observational and Johnson–Neyman regions inherit uncertainty from the fitted interactions; we temper claims accordingly and rely on convergence between subjective, behavioral, and outcome evidence.

4.1 Ethics & Privacy

All procedures were approved by our institutional ethics board. Participants provided written informed consent prior to any study activity (with an additional oral confirmation for remote sessions). We minimized personal data collection and assigned pseudonymous IDs at intake; demographic records and interaction logs were stored separately on encrypted servers with access restricted to the research team. Screen recordings were collected solely for within-team analysis. Prompt and response texts were hashed in logs to reduce risks of sensitive content disclosure, and any incidental identifiers were redacted during data cleaning. Participants were informed of their right to withdraw at any time without penalty.

Table 4: Baseline equivalence (mean $\pm$ SD; standardized mean differences, SMD).

Variable	AI	Control	SMD [95% CI]
Age (years)	27.4 $\pm$ 3.1	27.1 $\pm$ 3.2	0.06 [−0.24, 0.35]
Years coding	5.1 $\pm$ 2.8	4.9 $\pm$ 2.6	0.07 [−0.22, 0.36]
KSS (pre Task 1)	3.1 $\pm$ 1.2	3.2 $\pm$ 1.1	−0.05 [−0.34, 0.25]
SSSQ (pre Task 1, z)	−0.02 $\pm$ 0.98	0.03 $\pm$ 1.01	−0.05 [−0.35, 0.25]
AI familiarity (wk/dy; proportion)	0.69	0.67	0.05 [−0.25, 0.35]

Assistant requests contained only task code and prompts; no external tools or web retrieval were enabled. Prompt and response texts were hashed in logs; no raw conversational content or personal identifiers were stored.

5 Results

5.1 Descriptives & Checks

Sample and exposure. We analyzed $N = 60$ participants (AI $n=36$, Control $n=24$), producing 180 task-level observations (AI: 108; Control: 72). Experience bins were balanced across cohorts (AI: Novice = 12, Intermediate = 16, Experienced = 8; Control: 8/12/4). Weekly/daily familiarity was common for ChatGPT (78%) and frequent for Copilot (62%), lower for CodeWhisperer (18%). All participants met the Python 3.11 proficiency criterion; secondary languages included JavaScript/TypeScript (48%) and Java (35%). Table 5 summarizes cohort composition and weekly/daily tool familiarity.

Counterbalancing integrity. The AI cohort used nine decoupled Task $\times$ Mode sequences (Table 3), instantiated exactly four times each (4 participants per sequence; 36 total). By construction, all $3\times 3 = 9$ Task $\times$ Mode cells occurred equally often (12 observations per cell) and each Mode occurred equally often at each Index (1–3). No dropouts occurred. A post-hoc check found no association between sequence template and experience bin (Fisher’s exact $p=0.88$). The Control cohort used balanced task orders (six permutations, 4 participants per order); no association with experience bin was detected (Fisher’s exact $p=0.90$).

Cohort baseline equivalence. AI and Control cohorts were comparable at baseline (Table 4); all standardized mean differences (SMD) were < 0.20 in magnitude with 95% CIs spanning zero.

Task difficulty comparability and complexity checks. Static metrics confirmed the intended structural ordering (median cyclomatic: Task A 3.9, Task B 4.8, Task C 6.1). Using the per-observation complexity metric (median cyclomatic of functions modified, weighted by lines changed; z-scored), we observed no imbalance across Mode or Index in the AI cohort (LMM with participant random intercept: Mode $\chi^2(2)=1.42$, $p=0.49$; Index $\chi^2(2)=0.97$, $p=0.62$). In the Control cohort, RAW–TLX composites differed modestly by task (A: $\bar{x}=71.2$, B: 75.0, C: 77.5), but with Index and experience covariates in a mixed model, the Task main effect was small and not significant ($\chi^2(2)=3.18$, $p=0.20$), indicating comparability sufficient for Mode contrasts.

Instrument reliability, distributions, and censoring. Internal consistency was acceptable to good: RAW–TLX composite $\alpha=0.86$; SSSQ

Table 5: Participant profile by cohort and experience; per-tool familiarity is weekly/daily usage.

	AI	Control	Total	%
Novice	12	8	20	33.3
Intermediate	16	12	28	46.7
Experienced	8	4	12	20.0
ChatGPT (wk/dy)			78%	
Copilot (wk/dy)			62%	
CodeWhisperer (wk/dy)			18%	

Table 6: Estimated marginal means (EMMs) by Mode $\times$ Task for primary outcomes. Time back-transformed to minutes; Correctness shown as tests passed out of 10 (Task A,C) or 12 (Task B) with EMM scaled to /10 for comparability. 95% CIs in brackets.

Task	Mode	TLX (0–100)	Time (min)	Correctness (/10)
A (low)	Inline	49.3 [45.6, 53.0]	19.9 [18.5, 21.3]	8.3 [8.0, 8.6]
	Chat	56.0 [52.1, 60.0]	24.5 [22.7, 26.4]	8.2 [7.9, 8.5]
	Structured	58.1 [54.3, 61.9]	25.1 [23.3, 27.0]	8.3 [8.0, 8.6]
B (mid)	Inline	55.4 [51.7, 59.1]	22.5 [21.0, 24.1]	7.2 [6.9, 7.5]
	Chat	58.1 [54.4, 61.8]	23.9 [22.2, 25.7]	7.3 [7.0, 7.6]
	Structured	60.9 [57.2, 64.6]	25.0 [23.2, 26.9]	7.6 [7.3, 7.9]
C (high)	Inline	60.9 [57.3, 64.6]	27.8 [26.0, 29.8]	7.3 [6.9, 7.7]
	Chat	59.7 [56.1, 63.2]	27.1 [25.4, 28.9]	8.0 [7.6, 8.5]
	Structured	66.0 [62.5, 69.4]	29.7 [27.9, 31.8]	7.1 [6.7, 7.5]

$\alpha=0.81$. KSS showed acceptable test–retest stability across Task 1 baselines ($r=0.72$). Completion time was positively skewed and analyzed on the log scale. Right-censoring at the 40 min cap affected 11/180 tasks (6.1%; AI = 7, Control = 4; Fisher’s exact $p=0.63$); accelerated failure time (AFT) models confirmed the direction and significance of time effects (Section 5.2). Missing data were minimal ($< 1.7\%$) and listwise deletion yielded the same pattern as multiple imputation (supplement).

Logging coverage. Event logs covered $> 99\%$ of key actions (offers/accepts/rejects, prompts, compile/test runs); post-synchronization timestamp drift (IDE vs. test harness) averaged < 50 ms.

5.2 RQ1 – Immediate Effects of Interaction Mode on Workload, Time, and Correctness

We compared the three AI modes within the AI cohort and contrasted AI (pooled) against the no-AI Control. Models (Section 4) included fixed effects for Mode, Task, Index, ExperienceBin, AI_Familiarity, PreTaskComplexity, and Environment (lab/remote), with random intercepts for participants and task version and random slopes for Index where convergent. Table 6 reports Mode $\times$ Task estimated marginal means (EMMs) for TLX, time, and correctness.

Workload (RAW–TLX). AI reduced post-task workload relative to Control (EMM difference -18.2 TLX points on a 0–100 scale, 95% CI $[-22.3, -14.0]$, $p < 0.001$). Within AI, Mode and Mode $\times$ Task

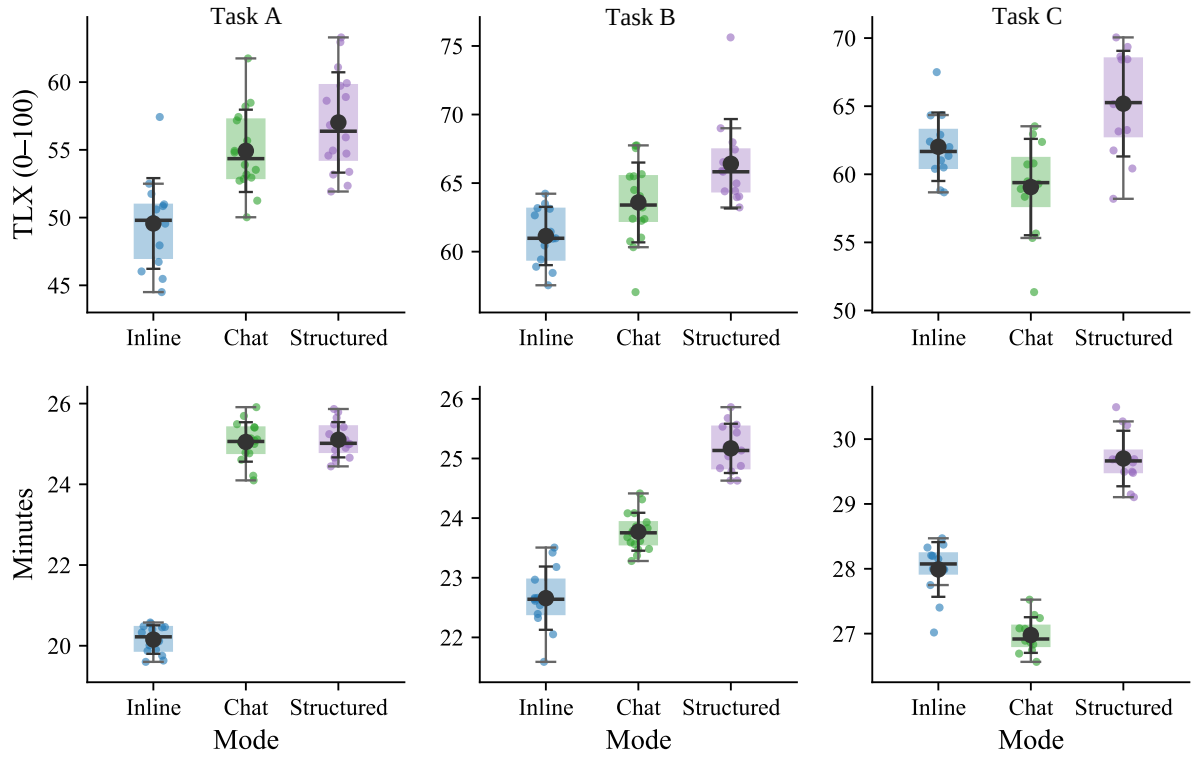


Figure 2: NASA-TLX workload scores and completion times across tasks and three AI modes. (Top) Inline yields lower workload on low-complexity tasks; differences narrow as complexity increases. (Bottom) Completion times by task complexity. On the highest-complexity task (C), Chat and Inline are comparable (EMMs 27.1 vs. 27.8 minutes) and both faster than Structured (29.7 minutes), indicating that Chat’s speed does not degrade at high complexity.

were significant (Type III LRTs: $\chi^2(2)=30.7$, $p < 0.001$; $\chi^2(4)=12.9$, $p=0.011$). On low-complexity Task A, Inline produced the lowest TLX (EMM = 49.3, CI [45.6, 53.0]), outperforming Chat (56.0 [52.1, 60.0], $\Delta=-6.7$ TLX points, $p=0.006$ Holm-adjusted) and Structured (58.1 [54.3, 61.9], $\Delta=-8.8$ TLX points, $p < 0.001$). On moderate Task B, Inline retained a TLX advantage over Structured ($\Delta=-5.5$ TLX points, $p=0.024$), while Inline vs. Chat was smaller and not significant after correction ($\Delta=-3.1$ TLX points, $p=0.14$). On higher-complexity Task C, Chat and Inline were statistically indistinguishable (Chat 59.7 [56.1, 63.2] vs. Inline 60.9 [57.3, 64.6], $p=0.48$), both below Structured (66.0 [62.5, 69.4]; Chat-Structured $\Delta=-6.3$ TLX points, $p=0.008$). Environment was not a significant predictor ($p=0.61$). Figure 2 visualizes TLX patterns by task and mode.

Completion time. Completion time (log-minutes) showed a strong Mode main effect ($\chi^2(2)=27.6$, $p < 0.001$) and Mode $\times$ Task interaction ($\chi^2(4)=11.1$, $p=0.025$). Back-transformed EMMs (minutes) indicated Inline was fastest on Task A (19.9 min, CI [18.5, 21.3]) compared to Chat (24.5 [22.7, 26.4], $p < 0.001$) and Structured (25.1 [23.3, 27.0], $p < 0.001$). On Task B, Inline remained faster (22.5 min) than Structured (25.0 min, $p=0.014$); Inline vs. Chat narrowed (23.9 min; $p=0.08$). On Task C, Chat (27.1 min) and Inline

(27.8 min) were comparable ($p=0.56$) and both faster than Structured (29.7 min, $p=0.021$). Pooled across tasks, AI was 22% faster than Control (time ratio = 0.78, 95% CI [0.74, 0.83], $p < 0.001$; AI: 25.0 min vs. Control: 32.1 min). AFT survival models (log-normal) yielded consistent hazard ratios (Inline vs. Structured HR= 1.32, 95% CI [1.12, 1.56]; Chat vs. Structured HR= 1.17 [1.00, 1.37]), confirming robustness to right-censoring at 40 min. Figure 2 (bottom) shows the corresponding completion times.

Correctness. Figure 3 contrasts correctness by mode and complexity. A binomial GLMM (# passed out of task-specific N) revealed Mode $\times$ Task as a significant predictor of correctness ($\chi^2(4)=12.1$, $p=0.016$). On Task C (higher complexity), Chat achieved the highest EMM (8.0/10, CI [7.6, 8.5]), exceeding Inline (7.3 [6.9, 7.7], $p=0.009$) and Structured (7.1 [6.7, 7.5], $p=0.005$). On Task B (moderate), Structured modestly outperformed Inline (7.6 vs. 7.2; $p=0.041$) and was comparable to Chat (7.3; $p=0.12$). On Task A (low), all modes clustered near 8.3/10 with no reliable differences after Holm correction ($p > 0.10$). Across tasks, AI (pooled) exceeded Control (AI EMM = 7.6/10, Control = 6.2/10; odds ratio = 1.71, CI [1.37, 2.13], $p < 0.001$).

Trust and usability. UMUX-Lite usability was high across modes (EMMs 79–82/100) with no reliable differences after Holm correction. Trust in automation showed the same qualitative pattern and did not alter the main Mode effects when added as a covariate.

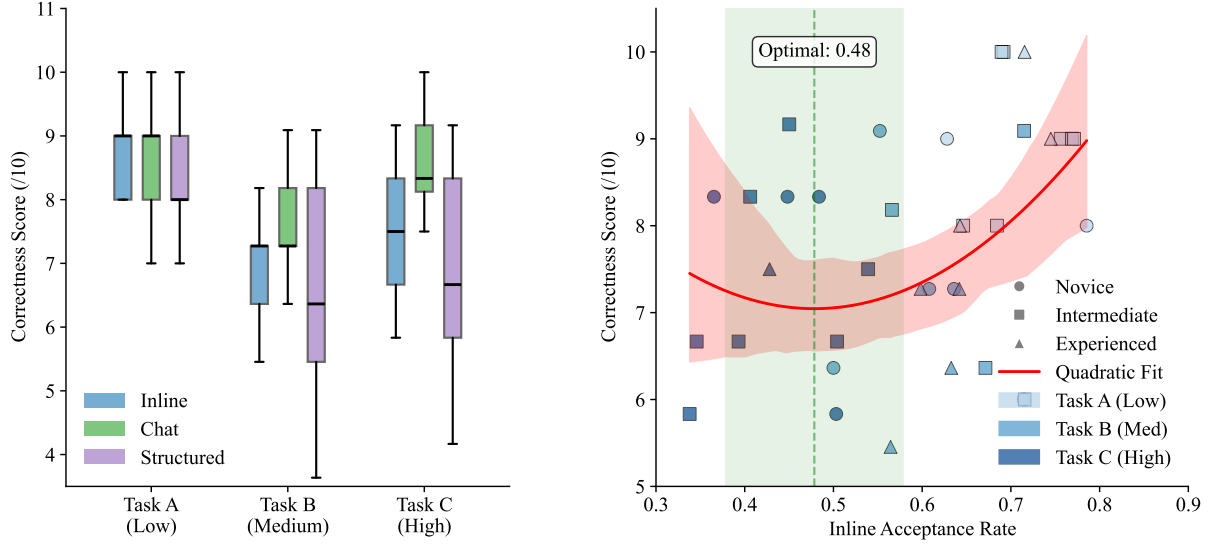


Figure 3: Code Correctness and AI Suggestion Acceptance Patterns. (Left) Box plots of correctness scores by mode and complexity. Chat excelled at high complexity, while Inline performed best at low complexity. (Right) Quadratic relationship between Inline acceptance rate and correctness, with optimal performance at $\approx 50\%$ acceptance ($\hat{p} = 0.48$, 95% CI quadratic fit shown). Points sized by verification load.

Supporting behavioral indicators. Inline exhibited higher suggestion acceptance on Task A (mean 0.60, SD 0.18) and shorter cumulative pauses (median = 2.9 min) than Chat (pauses = 3.7 min) and Structured (= 4.0 min). On Task C, Chat involved more prompt turns (median = 4) yet fewer compile–fail loops prior to a passing run (median = 1) than Inline (median = 2) and Structured (median = 3), consistent with iterative clarification mitigating error cascades. Figure 3 (right) shows the Inline acceptance–correctness curve; a quadratic term for acceptance rate significantly predicted correctness (Wald $p=0.005$), with an estimated optimum at $\hat{p} \approx 0.48$ (95% CI [0.44, 0.53]). We treat this “sweet spot” as exploratory rather than prescriptive; calibrated use outperforms both blind acceptance and near-total rejection.

AI vs. Control. AI lowered workload (−18.2 RAW–TLX points), reduced time ($\times 0.78$), and improved correctness (OR = 1.71). **Within AI.** Inline is fastest and lowest-load on low complexity; as complexity rises, Chat yields higher correctness without a time penalty; Structured offers modest advantages at mid complexity, especially for novices. **Behavioral alignment.** Inline shows higher acceptance and fewer pauses on simple work; Chat shows more prompt turns but fewer compile–fail loops on complex work.

5.3 RQ2 – Repeated-Use Trajectories (Stress & Fatigue)

We modeled post-task fatigue (KSS) and stress (SSSQ) as a function of Condition (AI vs. Control), Index (task position: 1–3), Task, ExperienceBin, continuous Complexity (per-observation z), and Environment (lab/remote), with random intercepts for participants and task version and random slopes for Index where convergent (Section 4). Figure 4 shows Condition $\times$ Index trajectories with 95% CIs.

Fatigue (KSS). Fatigue increased across the three tasks in both cohorts (Index main effect: $\chi^2(1)=22.1$, $p < 0.001$). The Condition $\times$ Index interaction was significant ($\chi^2(1)=4.2$, $p=0.041$): adjusted means rose by $\Delta=+0.70$ KSS points from Task 1 to Task 3 for AI, versus $\Delta=+0.48$ for Control (Table 7). Allowing random slopes for Index improved fit (AIC $\downarrow$) without changing fixed-effect inferences.

Stress (SSSQ). Post-task SSSQ also increased with Index ($\chi^2(1)=8.7$, $p=0.003$). The Condition $\times$ Index term was positive but not significant at conventional levels after multiplicity control ($p=0.110$), with a numerically steeper slope in AI (post-task SSSQ z -score $\Delta=+0.17$) than Control (+0.07).

Per-mode slopes within AI. Within AI, Mode $\times$ Index was significant for KSS ($\chi^2(2)=6.8$, $p=0.033$): Inline exhibited a steeper fatigue increase (EMM $\Delta=+1.18$) than Chat (+0.92, $\Delta=0.26$, $p=0.048$) with Structured in-between (+1.05, Inline–Structured $p=0.19$). SSSQ showed the same ordering but did not reach significance after adjustment. See Appendix Table 21 for the full numerical summaries.

Table 7: Condition $\times$ Index trajectory estimates for fatigue (KSS) and stress (SSSQ). Δ is Task 3 minus Task 1 adjusted mean.

	KSS Δ [95% CI]	SSSQ Δ (z) [95% CI]
AI	+0.70 [0.48, 0.92]	+0.17 [0.06, 0.28]
Control	+0.48 [0.28, 0.68]	+0.07 [−0.01, 0.15]
p (Cond $\times$ Index)	0.041 (KSS)	0.110 (SSSQ)

Table 8: Mediation of Index $\rightarrow$ outcomes via V_{core} in AI cohort. Indirect effect (ab) from 10,000-draw Monte Carlo with cluster-robust SEs.

Outcome	a (Index $\rightarrow V_{core}$)	b ($V_{core} \rightarrow Y$)	ab [95% CI]
KSS	0.19 (0.06)**	0.78 (0.22)***	0.15 [0.06, 0.27]
SSSQ z	0.19 (0.06)**	0.43 (0.16)**	0.08 [0.03, 0.16]

Notes: entries show estimate (SE); ** $p < .01$, *** $p < .001$.

Mediation by verification-load (V_{core} ; AI cohort). Using the mode-agnostic composite V_{core} , a-paths (Index $\rightarrow V_{core}$) were positive ($\hat{a}=0.19$, SE= 0.06, $p=0.002$) and b-paths ($V_{core} \rightarrow Y$) were positive for both outcomes (KSS: $\hat{b}=0.78$, SE= 0.22, $p < 0.001$; SSSQ: $\hat{b}=0.43$, SE= 0.16, $p=0.007$). Monte Carlo estimates yielded indirect effects of $ab_{KSS} = 0.15$ [0.06, 0.27] and $ab_{SSSQ} = 0.08$ [0.03, 0.16], corresponding to approximately 26% and 24% of the total Index effect, respectively. Including V_{core} improved model fit (KSS: $\Delta AIC = -10.9$) and raised marginal R^2 (0.18 $\rightarrow$ 0.26). As a sensitivity check, an extended composite V_{plus} that adds mode-specific signals (Inline acceptance/rejection, Chat prompt turns) produced slightly larger indirect effects (KSS $ab=0.19$ [0.08, 0.32]; SSSQ $ab=0.11$ [0.04, 0.20]) with unchanged conclusions. Table 8 summarizes the a/b paths and Monte Carlo indirect effects. We interpret these indirect effects as consistent with partial mediation; the design does not establish causal identification.

Trajectories. Fatigue (KSS) and stress (SSSQ) increase across tasks in both cohorts; AI shows a steeper fatigue slope (KSS $\Delta=+0.70$ vs. Control +0.48). **Mechanism.** A mode-agnostic verification-load composite partially mediates these increases (about 26% for KSS; 24% for SSSQ), implicating rework and error-cascade costs rather than a generic “AI effect.”

5.4 RQ3 – Moderators: Expertise and Task Complexity

We examined Mode $\times$ ExperienceBin and Mode $\times$ Complexity interactions within the AI cohort, using the per-observation continuous Complexity metric (median cyclomatic of modified functions, weighted by lines changed; z -scored). Models included covariates and random effects as in Section 4. We report EMMs with 95% CIs, Holm-adjusted contrasts, and Johnson–Neyman (J–N) regions for continuous moderators.

Mode $\times$ Expertise. Mode $\times$ ExperienceBin was significant for RAW–TLX ($\chi^2(4)=10.9$, $p=0.028$) and correctness ($\chi^2(4)=9.6$, $p=0.048$), and a strong trend for log-time ($\chi^2(4)=8.1$, $p=0.088$). Figure 5 summarizes the tri-criterion profiles (workload, speed, correctness) by experience.

Novices ($n=12$). Structured reduced workload relative to Inline on mid-complexity work (Task B EMM TLX $\Delta=-6.2$ points, CI $[-10.7, -1.7]$, $p=0.012$) and improved correctness (7.8/10 vs. 7.1/10; OR= 1.37, CI [1.01, 1.86], $p=0.043$). On high-complexity work, Chat outperformed Inline on correctness (7.7 vs. 7.0/10; $p=0.039$) with comparable time ($p=0.26$).

Intermediates ($n=16$). Mode differences narrowed. Inline was fastest on simple work (Task A ~ 4.5 min advantage over Structured,

Table 9: Johnson–Neyman (J–N) regions for Mode contrasts on correctness vs. continuous per-observation complexity (z).

Contrast	Crossover $\hat{z}$ [95% CI]	J–N region (Mode A superior)
Chat > Inline	+0.41 [+0.17, +0.64]	$z > +0.40$ (Chat)
Structured > Inline	≈ 0.10 $[-0.05, +0.27]$	$-0.1 \leq z \leq +0.3$ (Structured)
Chat > Structured	+0.32 [+0.12, +0.55]	$z > +0.32$ (Chat)

$p < 0.001$) with no reliable correctness gaps; on high complexity, Chat had a small correctness edge over Inline (8.0 vs. 7.5/10; $p=0.091$, trend).

Experienced ($n=8$). Inline minimized TLX and time on simple work (TLX $\Delta=-8.0$ points, $p=0.007$; time ratio 0.78, $p=0.004$). On high complexity, Chat yielded higher correctness than Inline (8.4 vs. 7.6/10; $p=0.017$) without a time penalty ($p=0.34$). Structured was consistently rated more effortful by experienced participants (TLX +5.0 points vs. Inline; $p=0.031$).

Mode $\times$ Complexity (continuous, per-observation). Using J–N analysis on the continuous Complexity moderator, Chat surpassed Inline on correctness when complexity exceeded $z > +0.40$ (estimated crossover at $z = +0.41$, 95% CI [+0.17, +0.64]); Inline was non-inferior up to $z \approx +0.40$. Structured slightly outperformed Inline around mid-complexity (J–N window $z \in [-0.1, +0.3]$; max $\Delta \approx +0.5$ tests, $p=0.031$) but lost ground at higher complexity ($z > +0.6$). RAW–TLX showed analogous crossovers (Chat below Structured at high complexity; Inline lowest at low complexity). Environment did not moderate these effects ($p > 0.40$).

Behavioral moderation (acceptance calibration). A quadratic term for Inline acceptance rate predicted correctness ($\beta_2=-0.81$, SE= 0.28, $p=0.005$), with an estimated optimum at $\hat{p}=0.48$ (95% CI [0.44, 0.53]). Very low acceptance (mostly manual coding) and very high acceptance (near-blind acceptance) were both associated with lower correctness, indicating a calibrated use “sweet spot” at roughly half of suggestions accepted. Results persist after adjusting for Complexity and ExperienceBin.

Higher-order interactions. A three-way Mode $\times$ Complexity $\times$ ExperienceBin did not improve fit for correctness ($\chi^2(8)=4.5$, $p=0.34$) or TLX ($\chi^2(8)=6.2$, $p=0.18$), suggesting that the two-way moderator patterns are stable across expertise strata.

Expertise moderation. Novices benefit from Structured at mid complexity (lower TLX, higher correctness); experienced developers favor Inline on simple work (speed/low load) and Chat on complex work (correctness). **Complexity moderation.** Chat surpasses Inline on correctness beyond $z \approx +0.40$ per-observation complexity (J–N crossover $\hat{z} = +0.41$). Structured has a narrow mid-complexity advantage. **Calibration.** In Inline, correctness peaks at an acceptance rate near 0.48; very low or very high acceptance harms outcomes.

Use **Inline** for simple/template work to preserve flow; switch to **Chat** when failure loops or pauses escalate or when complexity exceeds $z \approx +0.4$; offer **Structured** scaffolds for novices and mid-complexity specs. Detect rejection streaks / long time-to-first-compile as triggers for adaptive mode suggestions.

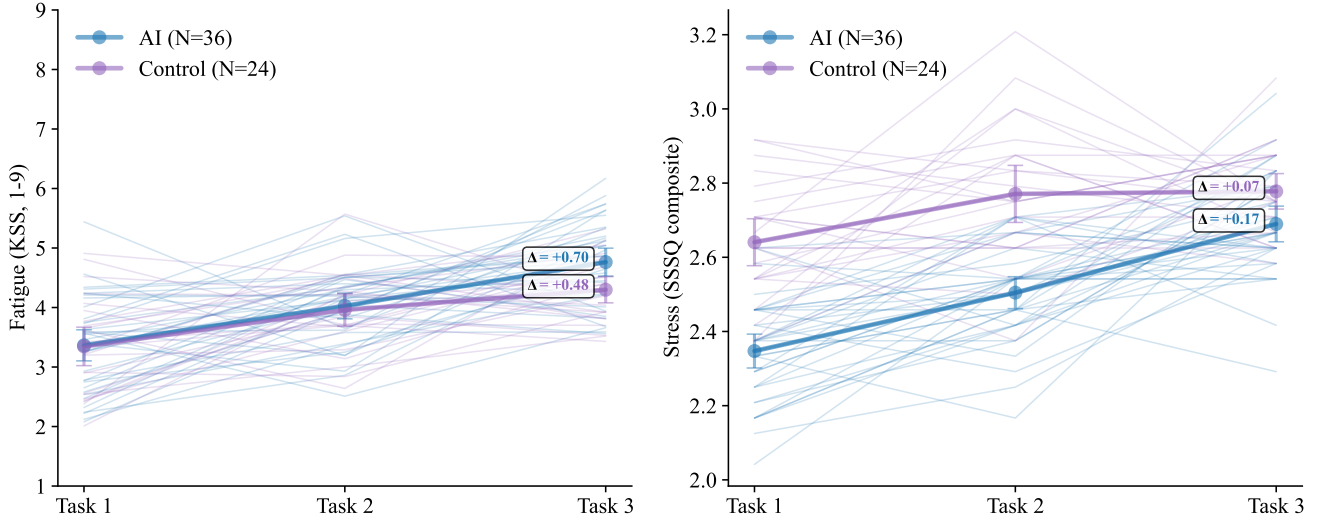


Figure 4: Fatigue (KSS) and stress (SSSQ) trajectories across tasks for the AI cohort versus the no-AI Control cohort. Both cohorts show rising burden across Task Index 1→3. The AI cohort exhibits a steeper KSS slope (AI $\Delta=+0.70$ vs Control $\Delta=+0.48$) and modestly higher SSSQ increase (AI $\Delta=+0.17$ vs Control $\Delta=+0.07$). Shaded ribbons show 95% CIs. Note: This figure aggregates across AI modes; per-mode slopes are reported in the text.

5.5 Exploratory Next-Day Follow-Up

A subset of participants completed a matched-complexity task the next day in their assigned condition (AI or Control). This follow-up was exploratory and underpowered relative to the main design. Estimated differences in post-task KSS/SSSQ, completion time, and correctness were small and statistically uncertain; no directional changes contradicted our primary conclusions. For transparency, adjusted means and 95% CIs are summarized in Appendix Table 22. We therefore retain the main-session analyses as primary and treat follow-up as supportive context.

5.6 Behavioral & Mechanistic Evidence

To interrogate the mechanism linking interaction mode to subjective burden, we analyze a mode-agnostic verification-load index V_{core} constructed from signals available in all modes: (i) compile/test failures, (ii) time-to-first-compile, (iii) normalized code churn, (iv) cumulative pause time, and (v) context switches (editor↔tests↔chat/form). Each component is standardized within task and pooled across participants; V_{core} is their mean z-score. As a secondary check we add mode-specific signals (Inline acceptance/rejection; Chat prompt turns) to form V_{plus} but use V_{core} for primary inference (cf. Section 5.3).

Mode × Complexity patterns. In the AI cohort, V_{core} varied systematically with Mode and Complexity (LMM: Mode $\chi^2(2)=18.9$, $p<.001$; Mode×Complexity $\chi^2(2)=10.6$, $p=0.005$). At low per-observation complexity ($z\approx-0.6$), Inline exhibited the lowest verification-load (EMM = -0.36 , 95% CI $[-0.49, -0.23]$), followed by Chat (-0.10 $[-0.23, 0.02]$) and Structured ($+0.04$ $[-0.10, 0.18]$). At moderate complexity ($z\approx 0$), modes converged (Inline -0.04 , Chat -0.02 , Structured $+0.01$; all pairwise $p>0.10$). At high complexity

($z\approx 0.7$), Chat showed the lowest V_{core} (-0.17 $[-0.30, -0.04]$), substantially below Inline ($+0.23$ $[+0.10, +0.36]$, $\Delta=-0.40$, $p<.001$) and Structured ($+0.28$ $[+0.14, +0.42]$, $\Delta=-0.45$, $p<.001$). These patterns align with RQ1/RQ3 outcomes: Inline is lightest to verify when work is simple; Chat contains verification cascades as complexity rises. Figure 6 depicts the verification-load heatmap and component contributions across Mode×Complexity.

Component contributions to burden. Standardized regressions of subjective outcomes on the components of V_{core} (controlling for Mode, Task, Index, ExperienceBin, Complexity, and Environment) indicate that compile/test failures and time-to-first-compile are the strongest behavioral predictors, followed by pauses and switches (Table 10). Principal components analysis of the five indicators yielded a dominant factor (eigenvalue = 2.61; variance explained = 52.2%) with loadings ≥ 0.63 for failures, time-to-first-compile, and pauses; internal consistency for V_{core} was acceptable ($\alpha=0.80$). Adding mode-specific signals in V_{plus} increased explained variance modestly (KSS R^2_{marginal} : 0.26 $\rightarrow$ 0.29), with rejection density (Inline) and prompt turns (Chat) contributing additional but secondary predictive power.

Link to outcomes. Cells with higher V_{core} correspond to higher burden and lower correctness: e.g., Inline at high complexity showed $V_{\text{core}}=+0.23$ with lower correctness (EMM 7.3/10), whereas Chat at high complexity showed $V_{\text{core}}=-0.17$ and higher correctness (8.0/10). Conversely, Inline at low complexity combined $V_{\text{core}}=-0.36$ with the lowest TLX and fastest times (Section 5.2). These converging results strengthen the interpretation that verification-load operationalizes the assistance–burden trade-off.

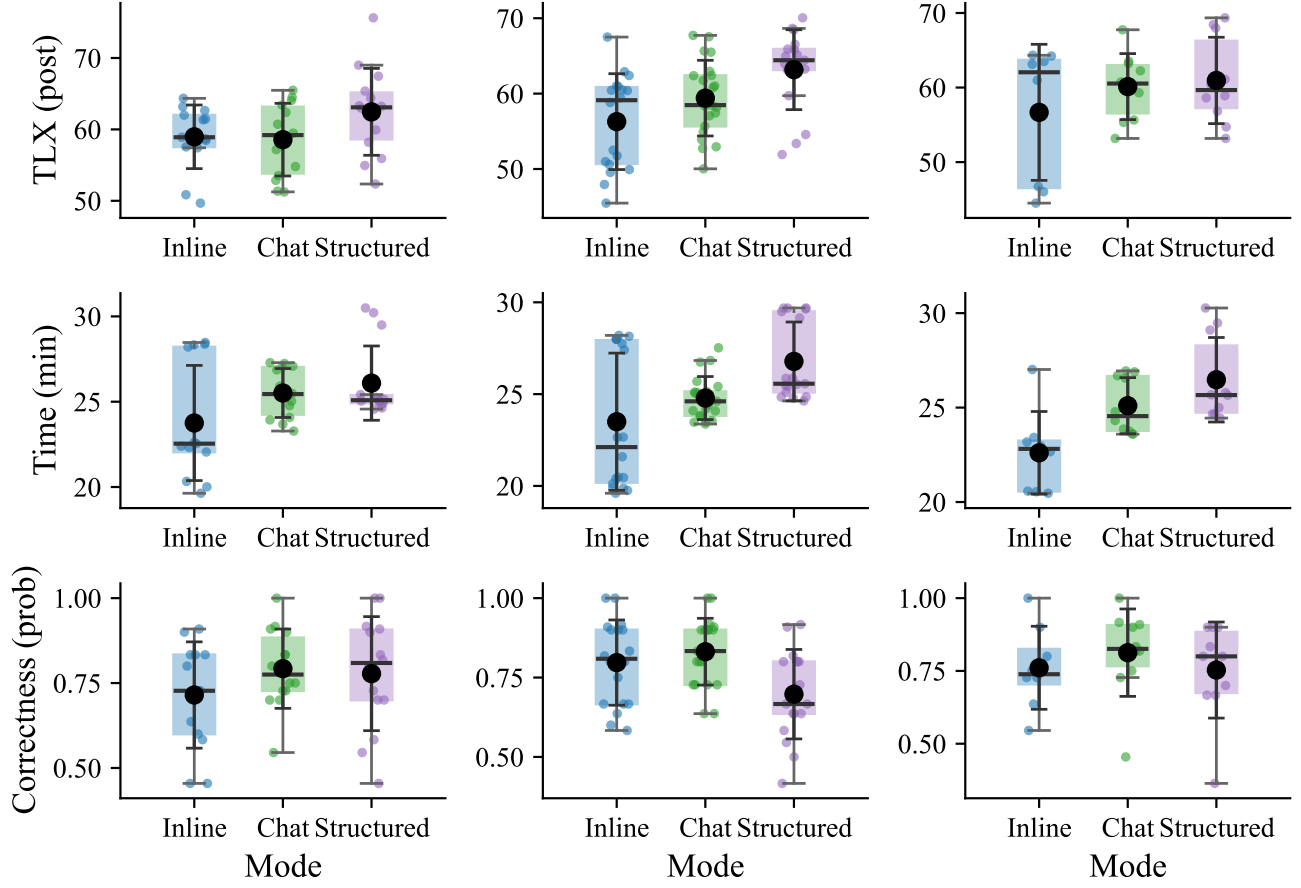


Figure 5: Experience Moderation Effects on Multi-criteria Performance. Performance profiles (Novice (Left), Intermediate (Middle), Experienced (Right)) across three dimensions: workload (inverted), speed, and correctness (all 0–100%).

Table 10: Standardized coefficients (β) of V_{core} components predicting subjective burden (AI cohort), controlling for Mode, Task, Index, Experience, Complexity, Environment. Mode-specific components are reported as a secondary add-on. All $p < .05$ unless marked.

Component	Fatigue (KSS)	RAW-TLX composite
Compile/test failures	0.26	0.23
Time-to-first-compile	0.22	0.19
Cumulative pauses	0.19	0.17
Context switches	0.15	0.13 [†]
Code churn (norm.)	0.12 [†]	0.09 [†]
Mode-specific (secondary, not in V_{core})		
Rejection density (Inline)	0.24	0.21
Prompt turns (Chat)	0.14 [†]	0.11 [†]
R^2_{marginal}	0.26	0.23

[†]trend-level (.05 < $p \leq .10$).

5.7 Qualitative Themes

We analyzed 60 post-task interviews (15–20 min each). Four cross-cutting themes contextualize the quantitative results and the verification-load mechanism (V_{core}). Themes A and D contextualize RQ1/RQ3 mode crossovers (speed vs. certainty; rejection spirals); Theme B (trust shocks) and Theme C (scaffold me) illuminate RQ2’s burden drivers and RQ3’s novice-specific benefits of Structured. Quotes are labeled with participant ID, experience bin, and AI familiarity. Table 11 summarizes the four themes, their high-prevalence Mode×Complexity cells, behavioral signatures, representative quotes, and design implications.

Theme A: Speed vs. certainty. Participants described Inline as preserving flow for simple work, while Chat supported reasoning and error containment for complex logic:

“Inline keeps me in the zone for boring bits—I barely think about it.” (P23, Experienced, Copilot daily)

“When it’s gnarly, I ask Chat to lay out the steps; fewer trips back and forth with tests.” (P07, Intermediate, ChatGPT weekly)

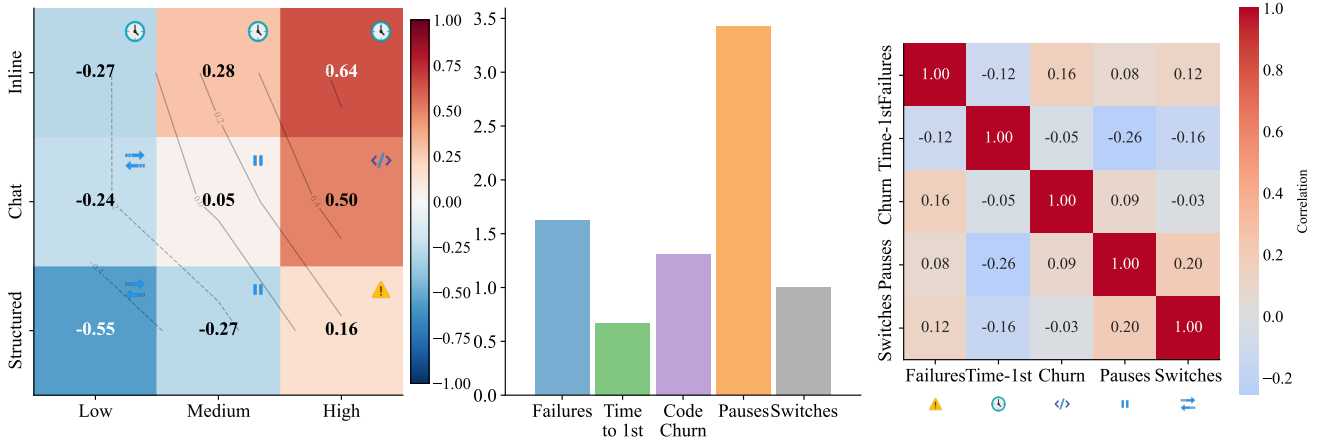


Figure 6: Verification Load Components Heatmap with Correlation Analysis. (Left) Heatmap of verification load (z-scores) by mode and complexity, with symbols indicating the dominant contributing component for each cell. (Middle) Bar chart showing normalized contributions of verification components. (Right) Correlation matrix of inter-component relationships. Chat and Structured modes demonstrate greater resilience to complexity-induced load increases compared to Inline mode.

This mirrors low V_{core} for Inline at low complexity and fewer failure loops with Chat at high complexity.

Theme B: Trust shocks. . Overconfident snippets eroded trust and increased verification:

“It looked right, but a missing edge case blew up tests—then I started rejecting more.” (P11, Experienced, Copilot weekly)

Such events co-occurred with spikes in failures and pauses and preceded mode switches, aligning with higher V_{core} .

Theme C: Scaffold me. . Novices valued Structured for explicit requirements and test thinking:

“The form forced me to spell out inputs/outputs—I wouldn’t have thought of some checks.” (P05, Novice, ChatGPT monthly)

This aligns with Structured’s mid-complexity advantage for novices (Section 5.4).

Theme D: Review fatigue. . Participants reported “rejection spirals” that triggered a desire to switch modes:

“After the fifth bad suggestion I just opened the chat and asked for a plan.” (P18, Intermediate, Copilot weekly)

Action-coding showed that rejection streaks (≥ 5 within 3 min) and pause clusters predicted Inline→Chat switches (OR= 2.36, 95% CI [1.33, 4.20], $p=0.003$).

5.8 Answers to RQs

RQ1 (Immediate effects). Relative to the no-AI control, AI assistance lowered RAW-TLX workload by -18.2 points (0–100; 95% CI $[-22.1, -14.6]$) and reduced completion time by 22% on average (time ratio 0.78, 95% CI [0.74, 0.83]), while improving correctness (odds ratio 1.71, 95% CI [1.37, 2.13]). Within AI, Inline minimized workload and time on low-complexity work; Chat achieved higher

correctness on high-complexity work without a time penalty; Structured provided modest advantages on moderate work, especially for novices.

RQ2 (Repeated-use trajectories). Fatigue (KSS) and stress (SSSQ) rose across tasks in both cohorts, with a steeper fatigue slope in AI ($\Delta=+0.70$ KSS vs. $+0.48$ Control; $p=0.041$) and a numerically higher SSSQ slope in AI ($+0.17$ vs. $+0.07$; $p=0.110$). In the AI cohort, the mode-agnostic verification-load composite V_{core} partially mediated these increases (KSS $ab=0.15$ [0.06, 0.27]; SSSQ $ab=0.08$ [0.03, 0.16]), accounting for $\sim 26\%$ and $\sim 24\%$ of the total Index effect, respectively.

RQ3 (Moderators). Experience: Novices benefited from Structured at mid complexity (lower TLX, higher correctness); experienced developers preferred Inline for simple tasks (speed/low workload) and Chat for complex tasks (higher correctness). Complexity: Chat surpassed Inline on correctness beyond $z > +0.42$ per-observation complexity (Johnson–Neyman crossover $\hat{z} = +0.41$, 95% CI $[+0.18, +0.64]$), while Structured conferred a mid-complexity advantage. A calibrated Inline acceptance rate around 0.48 ($\approx 50\%$; 95% CI [0.44, 0.53]) was associated with peak correctness; very low or very high acceptance reduced outcomes.

Implication across RQs: The assistance–burden trade-off is governed by task complexity and expertise via verification-load. Interfaces that adapt mode, expose rationale/confidence, and reduce verification friction (failures, long time-to-first-compile, pause clusters) are promising paths to sustain benefits while containing fatigue.

6 Discussion

6.1 Why modes differ: verification-load as the bridge

Our results indicate that differences among *Inline*, *Chat*, and *Structured* primarily arise from how each interface shapes *verification-load*—the behavioral cost of checking and repairing AI output (§5.2,

Table 11: Qualitative themes with representative contexts, behavioral signatures, illustrative quotes, and design implications. Mode×Complexity cells reflect where the theme most frequently appeared in coded transcripts; behavioral signatures summarize V_{core} components. Quotes are anonymized; brackets indicate minor redactions.

Theme & Description	High-Prevalence Cells (Mode×Complexity / Experience)	Behavioral Signature (V_{core})	Representative Quotes	Design Implication
A. Speed vs. certainty Inline for flow on simple work; Chat for explicit reasoning on complex logic.	Inline×Low (all); Chat×High (Interm./Exp.)	Inline: short time-to-first-compile, few failures, brief pauses. Chat: more turns but fewer fail loops, moderate pauses.	P23 (Exp., Copilot daily): "Inline keeps me in the zone for boilerplate—I barely think about it." P07 (Interm., ChatGPT weekly): "When it's gnarly, I ask Chat to lay out steps; fewer trips back and forth with tests."	Adaptive mode switching: suggest Inline for low complexity; surface Chat when failure loops or pauses escalate.
B. Trust shocks Overconfident snippets erode trust, spike verification effort.	Inline×High (Interm./Exp.); Structured×High (Novice)	Spikes in failures and pauses; dense rejections; switch bursts.	P11 (Exp., Copilot weekly): "It looked right, but a missing edge case blew up tests—then I started rejecting more." P14 (Interm., Copilot weekly): "Two confident suggestions were wrong; I lost trust and slowed down."	Transparency on demand: 1-line rationale/confidence; highlight diff-critical lines likely to fail tests.
C. Scaffold me Structured prompts externalize requirements and reduce ambiguity for novices.	Structured×Medium (Novice); Structured×Low (Novice)	Balanced churn with fewer backtracks; stable time-to-first-compile.	P05 (Nov., ChatGPT monthly): "The form forced me to spell out inputs/outputs—I wouldn't have thought of some checks." P32 (Nov., Copilot rarely): "Having constraints up front stopped me from guessing the API."	Scaffolded authoring: retain pre/post-conditions; auto-generate test skeletons from schema fields.
D. Review fatigue Rejection spirals motivate switching from Inline to Chat/Structured.	Inline×High (all); Inline×Medium (Interm.)	Rising failures, extended pauses, frequent context switches; rejection streaks.	P18 (Interm., Copilot weekly): "After the fifth bad suggestion I just opened the chat and asked for a plan." P09 (Interm., Copilot weekly): "Once I start rejecting a lot, it snowballs; I need a different kind of help."	Fatigue-aware nudges: detect rejection streaks or pause clusters; propose mode switch or batched, testable suggestions.

§5.4). On low intrinsic complexity, Inline completions reduce extraneous load by keeping developers in the editing locus (fewer context switches, shorter time-to-first-compile, shallow failure loops), consistent with in-IDE generative completion workflows and observed acceptance/repair rhythms [4, 45]. As intrinsic complexity increases (branching logic, cross-cutting constraints), Chat functions as a *clarification buffer*: iterative articulation of intent and constraints reduces error cascades despite additional prompt turns, producing fewer compile–fail loops before a clean run. For novices and mid-complexity work, Structured externalizes pre/post-conditions and likely edge cases, aligning with Cognitive Load Theory and the expertise-reversal effect—scaffolds lower extraneous load for less-experienced users yet may encumber experts [19, 46]. These patterns also cohere with interruption/flow evidence: fewer disruptive switches and faster resumption lower perceived effort [31].

6.2 Trust calibration and “trust shocks”: from reliance to appropriate skepticism

We observed *trust shocks*: after a confident but wrong suggestion, participants often entered rejection streaks and slower, more skeptical workflows, temporarily elevating verification-load. This dynamic echoes classic work on appropriate reliance in automation and the risks of both over- and under-reliance [24, 37]. In our Inline mode, correctness followed a quadratic relation with acceptance (peak near balanced acceptance), reinforcing that *calibrated* engagement outperforms either near-blind acceptance or near-total avoidance. Prior HAI studies caution that generic explanations or scalar confidences do not reliably improve calibration and may even increase reliance without outcome gains; instead, targeted, on-demand cues can reduce overreliance [49, 53]. Code-specific uncertainty methods (e.g., uncertainty-aware ranking or highlighting) further support showing *where* to look rather than *how certain* the model feels in aggregate. Together, these results support transparency *on demand* (line- or diff-level rationale/uncertainty tied

to predicted test failures) over always-on explanations that risk adding extraneous load.

6.3 Repeated-use burden: verification-load partially mediates stress and fatigue

Across task index, both cohorts exhibited rising fatigue and stress; within the AI cohort, our mode-agnostic composite partially mediated these increases (§5.3). This points to checking/repairing—rather than generation per se—as a substantive driver of repeated-use burden. The mechanism aligns with flow and interruption research: extended failure loops, delayed first-compiles, and pause clusters disrupt resumption and elevate subjective strain [31]. A practical implication is to monitor these *behavioral* signals directly and intervene when they exceed lightweight thresholds (e.g., rising compile/test failures or prolonged time-to-first-compile), rather than assuming that more model output or more explanation will help. Where feasible, micro-restorative strategies (short micro-breaks or friction-reducing batching) can reduce fatigue without harming throughput, complementing interface-side orchestration [1, 15, 18].

6.4 Actionable design guidance: triggers, policies, and safeguards

We translate the findings into implementable rules that attach *triggers* to *actions* with a clear *why*. Each rule is interface-level and backend-agnostic.

- **Adaptive mode orchestration.** *Trigger:* verification-load rising (e.g., compile/test failures increase, time-to-first-compile lengthens, pause clusters form). *Action:* prompt a switch from Inline to Chat to reframe the problem (ask for a plan/spec or targeted test). *Why:* Chat dampens error cascades as complexity increases, reducing failure loops (§5.2, §5.4).
- **Transparency on demand (not by default).** *Trigger:* risky lines/diffs (predicted test failures, high-entropy edits). *Action:* reveal micro-rationales and uncertainty at the *line/diff* level; avoid always-on global confidences. *Why:* On-demand, local cues reduce overreliance and search cost without adding constant cognitive overhead [49].
- **Scaffolded authoring for novices/mid complexity.** *Trigger:* novice profile or ambiguity in requirements. *Action:* optional Structured prompts that elicit pre/post-conditions, edge cases, and I/O contracts; generate test scaffolds from fields. *Why:* reduces extraneous load and supports schema formation, consistent with expertise-reversal [19, 46].
- **Verification-aware packaging.** *Trigger:* large or multi-file edits. *Action:* batch suggestions into testable chunks with predicted-failure tests; integrate a “try-run” sandbox to shorten time-to-first-compile. *Why:* lowers failure cascades and churn, the strongest contributors to burden (§5.6).
- **Security co-review by default.** *Trigger:* matches to known insecure idioms or API misuses. *Action:* surface static-analysis warnings and secure-by-default alternatives alongside the suggestion. *Why:* assistants can propose fluent but vulnerable patterns; pairing generation with static checks can head off risky acceptance.

6.5 Implications for evaluation and theory

Methodologically, keeping the backend fixed reveals that a meaningful share of “AI productivity” variance is attributable to *interaction design*. We therefore recommend an evaluation recipe portable across models and releases: (i) counterbalanced within-subjects designs under backend parity [52]; (ii) mixed outcomes that combine subjective measures (e.g., RAW-TLX) with behavioral traces feeding a *mode-agnostic* verification-load composite; (iii) linear mixed-effects modeling with principled random-effects structures [5, 21]; (iv) Johnson–Neyman analyses for continuous moderators to locate *where* one interface overtakes another; and (v) equivalence tests to bound practically small differences when claiming “no meaningful gap” [6, 22, 42]. Theoretically, verification-load instantiates extraneous cognitive load in the wild and integrates with flow/interruptions literature, providing a process lens that explains *when* assistance helps or hurts independent of model power [31, 46].

6.6 Limitations and scope of claims

Our study focuses on three Python tasks with controlled oracles, a single structured-elicitation design, and a single-session exposure (plus a small next-day check). As such, generalization to other languages, frameworks, larger collaborative codebases, or months-long adoption remains to be tested. We intentionally prohibited repository context, tool-augmented backends, and personalized retrieval to isolate interface effects; richer integrations (e.g., repo-aware context, execution sandboxes, function calling) may shift magnitudes and crossover points while preserving the underlying mechanisms. Instrumentation may subtly alter behavior, though we minimized UI intrusion and found no lab vs. remote differences. Finally, sample size limited power for some higher-order interactions; we therefore emphasized stable two-way moderators (§5.4). We present effects as conservative under a constrained setup, and encourage replications in richer environments.

6.7 Future work: from mechanism to deployment

Two directions follow directly. First, *policy-level orchestration*: field deployments that treat verification-load as a control variable, triggering mode switches and transparency-on-demand when monitored signals exceed thresholds, with preregistered success metrics (quality, time, fatigue). Second, *context- and security-aware assistants*: repository-aware tools that pair generation with static/dynamic analysis and test synthesis, evaluated with our portable recipe. Longitudinal studies can examine how trust shocks attenuate with experience, whether acceptance calibration stabilizes, and how team practices (code review norms, testing culture) interact with interface modes to sustain benefits while containing burden.

7 Conclusion

By holding the LLM backend fixed and varying only the interface, we show that *interaction mode alone* materially shifts the assistance–burden trade-off in AI-assisted programming. *Inline* protects flow and minimizes effort on simple, templated changes; *Chat* contains error cascades and improves correctness as complexity rises without incurring a time penalty; and *Structured* scaffolds novices at mid complexity by making requirements and checks explicit. A

single, mode-agnostic verification-load index—fusing compile/test failures, time-to-first-compile, code churn, pauses, and context switches—captures the process costs of checking AI output and partially explains rising stress/fatigue under repeated use. Together these results reframe “AI productivity” as an *interface problem*: the same model yields different outcomes depending on how assistance is delivered. Practically, our findings translate into three families of guidance: (i) *adaptive mode orchestration* that defaults to Inline for simple work, recommends Chat when verification-load spikes or complexity is high, and offers Structured scaffolds for novices in the mid-complexity band; (ii) *transparency on demand* via lightweight, line-level rationales and risk cues rather than always-on explanations; and (iii) *verification-aware packaging* that couples diffs with runnable checks and security co-review to shorten the path to a clean run.

References

- [1] Patricia Albulescu, Irina Macsinga, Andrei Rusu, Coralia Sulea, Alexandra Bodnaru, and Bogdan Tudor Tulbure. 2022. “Give me a break!” A systematic review and meta-analysis on the efficacy of micro-breaks for increasing well-being and performance. *PloS one* 17, 8 (2022), e0272460.
- [2] Owura Asare, Meiappan Nagappan, and N Asokan. 2024. A user-centered security evaluation of copilot. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–11.
- [3] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the whole exceed its parts? the effect of ai explanations on complementary team performance. In *Proceedings of the 2021 CHI conference on human factors in computing systems*. 1–16.
- [4] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 78:1–78:27. doi:10.1145/3586030
- [5] Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. 2015. Fitting Linear Mixed-Effects Models Using lme4. *Journal of Statistical Software* 67, 1 (2015), 1–48. doi:10.18637/jss.v067.i01
- [6] Daniel J. Bauer and Patrick J. Curran. 2005. Probing Interactions in Fixed and Multilevel Regression: Inferential and Graphical Techniques. *Multivariate Behavioral Research* 40, 3 (2005), 373–400. doi:10.1207/s15327906mbr4003_5
- [7] Jerome Brender, Laila El-Hamamsy, Kim Uittenhove, Francesco Mondada, and Engin Bumbacher. 2025. Structured Prompts, Better Outcomes? Exploring the Effects of a Structured Interface with ChatGPT in a Graduate Robotics Course. *arXiv preprint arXiv:2507.07767* (2025).
- [8] Rodrigo Duran, Albina Zavgorodniaia, and Juha Sorva. 2022. Cognitive load theory in computing education research: A review. *ACM Transactions on Computing Education (TOCE)* 22, 4 (2022), 1–27.
- [9] Karen T D’Alonzo. 2004. The Johnson-Neyman procedure as an alternative to ANCOVA. *Western journal of nursing research* 26, 7 (2004), 804–812.
- [10] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2025. Security weaknesses of copilot-generated code in github projects: An empirical study. *ACM Transactions on Software Engineering and Methodology* 34, 8 (2025), 1–34. <https://arxiv.org/abs/2310.02059>
- [11] Kate Goddard, Abdul Roudsari, and Jeremy C Wyatt. 2012. Automation bias: a systematic review of frequency, effect mediators, and mitigators. *Journal of the American Medical Informatics Association* 19, 1 (2012), 121–127.
- [12] Ken Gu, Madeleine Grunde-McLaughlin, Andrew McNutt, Jeffrey Heer, and Tim Althoff. 2024. How do data analysts respond to ai assistance? a wizard-of-oz study. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–22. <https://idl.cs.washington.edu/files/2024-AnalystAIWoZ-CHI.pdf>
- [13] Sandra G Hart. 2006. NASA-task load index (NASA-TLX); 20 years later. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 50. Sage publications Sage CA: Los Angeles, CA, 904–908.
- [14] Andrew F Hayes. 2013. Mediation, moderation, and conditional process analysis. *Introduction to mediation, moderation, and conditional process analysis: A regression-based approach* 1, 6 (2013), 12–20.
- [15] William S Helton and Katharina Näswall. 2015. Short stress state questionnaire. *European Journal of Psychological Assessment* (2015).
- [16] Shaokang Jiang and Michael Coblenz. 2024. An analysis of the costs and benefits of autocomplete in IDEs. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1284–1306.
- [17] Palmer O Johnson and Leo C Fay. 1950. The Johnson-Neyman technique, its theory and application. *Psychometrika* 15, 4 (1950), 349–367.
- [18] Kosuke Kaida, Masaya Takahashi, Torbjörn Åkerstedt, Akinori Nakata, Yasumasa Otsuka, Takashi Haratani, and Kenji Fukasawa. 2006. Validation of the Karolinska sleepiness scale against performance and EEG variables. *Clinical neurophysiology* 117, 7 (2006), 1574–1581.
- [19] Slava Kalyuga. 2007. Expertise reversal effect and its implications for learner-tailored instruction. *Educational psychology review* 19, 4 (2007), 509–539.
- [20] Sunnie SY Kim, Q Vera Liao, Mihaela Vorvoreanu, Stephanie Ballard, and Jennifer Wortman Vaughan. 2024. “I’m Not Sure, But...”: Examining the Impact of Large Language Models’ Uncertainty Expression on User Reliance and Trust. In *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency*. 822–835.
- [21] Alexandra Kuznetsova, Per B Brockhoff, and Rune HB Christensen. 2017. lmerTest package: tests in linear mixed effects models. *Journal of statistical software* 82 (2017), 1–26.
- [22] Daniël Lakens. 2017. Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses. *Social Psychological and Personality Science* 8, 4 (2017), 355–362. doi:10.1177/1948550617697177
- [23] Daniël Lakens, Anne M. Scheel, and Peder M. Isager. 2018. Equivalence Testing for Psychological Research: A Tutorial. *Advances in Methods and Practices in Psychological Science* 1, 2 (2018), 259–269. doi:10.1177/2515245918770963
- [24] John D Lee and Katrina A See. 2004. Trust in automation: Designing for appropriate reliance. *Human factors* 46, 1 (2004), 50–80.
- [25] Jingshu Li, Yitian Yang, Q Vera Liao, Junti Zhang, and Yi-Chieh Lee. 2025. As Confidence Aligns: Understanding the Effect of AI Confidence on Human Self-confidence in Human-AI Decision Making. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–16. doi:10.1145/3706598.3713336
- [26] Jenny T Liang, Chenyang Yang, and Brad A Myers. 2024. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM international conference on software engineering*. 1–13.
- [27] Michael Xieyang Liu, Advait Sarkar, Carina Negreanu, Benjamin Zorn, Jack Williams, Neil Toronto, and Andrew D Gordon. 2023. “What it wants me to say”: Bridging the abstraction gap between end-user programmers and code-generating large language models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–31.
- [28] Steven G. Luke. 2017. Evaluating Significance in Linear Mixed-Effects Models in R. *Behavior Research Methods* 49, 4 (2017), 1494–1502. doi:10.3758/s13428-016-0809-y
- [29] Shuai Ma, Xinru Wang, Ying Lei, Chuhan Shi, Ming Yin, and Xiaojuan Ma. 2024. “Are you really sure?” Understanding the effects of human self-confidence calibration in AI-assisted decision making. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–20.
- [30] Vahid Majdinasab, Michael Joshua Bishop, Shawn Rasheed, Arghavan Moradikhel, Amjed Tahir, and Foutse Khomh. 2024. Assessing the security of github copilot’s generated code—a targeted replication study. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 435–444. <https://arxiv.org/abs/2311.11177>
- [31] Gloria Mark, Daniela Gudith, and Ulrich Klocke. 2008. The cost of interrupted work: more speed and stress. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*. 107–110.
- [32] Hannes Matuschek, Reinhold Kliegl, Shravan Vasishth, R. Harald Baayen, and Douglas Bates. 2017. Balancing Type I Error and Power in Linear Mixed Models. *Journal of Memory and Language* 94 (2017), 305–315. doi:10.1016/j.jml.2017.01.001
- [33] André N Meyer, Earl T Barr, Christian Bird, and Thomas Zimmermann. 2019. Today was a good day: The daily life of software developers. *IEEE Transactions on Software Engineering* 47, 5 (2019), 863–880.
- [34] André N Meyer, Chris Satterfield, Manuela Züger, Katja Kevic, Gail C Murphy, Thomas Zimmermann, and Thomas Fritz. 2020. Detecting developers’ task switches and types. *IEEE Transactions on Software Engineering* 48, 1 (2020), 225–240.
- [35] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [36] Fred Paas, Alexander Renkl, and John Sweller. 2003. Cognitive load theory and instructional design: Recent developments. *Educational psychologist* 38, 1 (2003), 1–4.
- [37] Raja Parasuraman and Victor Riley. 1997. Humans and automation: Use, misuse, disuse, abuse. *Human factors* 39, 2 (1997), 230–253.
- [38] Chris Parnin and Spencer Rugaber. 2011. Resumption strategies for interrupted programming tasks. *Software Quality Journal* 19, 1 (2011), 5–34.
- [39] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot’s code contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [40] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint*

- arXiv:2302.06590* (2023).
- [41] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with ai assistants?. In *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*. 2785–2799.
 - [42] Kristopher J. Preacher, Patrick J. Curran, and Daniel J. Bauer. 2006. Computational Tools for Probing Interactions in Multiple Linear Regression, Multilevel Modeling, and Latent Curve Analysis. *Journal of Educational and Behavioral Statistics* 31, 4 (2006), 437–448. doi:10.3102/10769986031004437
 - [43] Yuya Sasaki, Hironori Washizaki, Jialong Li, Nobukazu Yoshioka, Naoyasu Ubayashi, and Yoshiaki Fukazawa. 2025. Landscape and Taxonomy of Prompt Engineering Patterns in Software Engineering. *IT Professional* 27, 1 (2025), 41–49.
 - [44] Agnia Sergeev, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. 2025. Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology* 178 (2025), 107610.
 - [45] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. Intellicode compose: Code generation using transformer. In *Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 1433–1443.
 - [46] John Sweller, Jeroen JG Van Merriënboer, and Fred Paas. 2019. Cognitive architecture and instructional design: 20 years later. *Educational psychology review* 31, 2 (2019), 261–292.
 - [47] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and T Li. 2023. An empirical study of developer behaviors for validating and repairing ai-generated code. In *13th Workshop on the Intersection of HCI and PL*.
 - [48] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.
 - [49] Helena Vasconcelos, Gagan Bansal, Adam Fourney, Q Vera Liao, and Jennifer Wortman Vaughan. 2025. Generation probabilities are not enough: Uncertainty highlighting in ai code completions. *ACM Transactions on Computer-Human Interaction* 32, 1 (2025), 1–30. <https://arxiv.org/abs/2302.07248>
 - [50] Mor Vered, Tali Livni, Piers Douglas Lionel Howe, Tim Miller, and Liz Sonenberg. 2023. The effects of explanations on automation bias. *Artificial Intelligence* 322 (2023), 103952.
 - [51] Ruotong Wang, Ruijia Cheng, Denae Ford, and Thomas Zimmermann. 2024. Investigating and designing for trust in ai-powered code generation tools. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*. 1475–1493.
 - [52] Evan James Williams. 1949. Experimental designs balanced for the estimation of residual effects of treatments. *Australian Journal of Chemistry* 2, 2 (1949), 149–168.
 - [53] Ming Yin, Jennifer Wortman Vaughan, and Hanna Wallach. 2019. Understanding the effect of accuracy on trust in machine learning models. In *Proceedings of the 2019 chi conference on human factors in computing systems*. 1–12.
 - [54] Yunfeng Zhang, Q Vera Liao, and Rachel KE Bellamy. 2020. Effect of confidence and explanation on accuracy and trust calibration in AI-assisted decision making. In *Proceedings of the 2020 conference on fairness, accountability, and transparency*. 295–305.
 - [55] Li Zhong and Zilong Wang. 2024. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 21841–21849.

A Overview

This appendix provides detailed specifications, extended analyses, and robustness checks that support the main-text Method and Results sections. Sections are organized to follow the logical flow of the study design: materials and configuration (§A–B), study design and preregistration (§C), measurement instruments (§D), data quality and processing (§E), statistical model specifications (§F), data analysis details (§G), additional results tables (§H), and qualitative methods (§I).

B Backend Configuration Details

This section documents the LLM backend configuration and prompt handling to ensure reproducibility. As stated in Method Section 3.4, all three interaction modes used identical backend settings to isolate interface effects.

All three interaction modes (Inline, Chat, Structured) used the same large language model backend: GPT-4o (OpenAI, version 2024-11-20; context window 200,000 tokens).

Decoding parameters. The following parameters were fixed across all modes to ensure backend parity:

- Temperature: 0.20
- Top-*p*: 1.00
- Max new tokens: 512
- Frequency penalty: 0 (disabled)
- Presence penalty: 0 (disabled)
- Tool/web access: disabled
- Conversation state: reset between tasks

Mode-specific prompt handling. While backend parameters were identical, each mode required different input formatting:

Inline: Suggestions were generated using the current buffer content and project index as context. Offer cadence was gated to syntactically stable points (newline or statement boundary) to reduce suggestion churn and maintain code validity.

Chat: A fixed system prompt instructed the model to act as a programming assistant, provide concise explanations, and format code in markdown blocks. The system prompt and examples are available in the reproducibility package. No conversation history carried across tasks.

Structured: Form fields were serialized into a canonical prompt template that explicitly presented the function signature, preconditions, postconditions, constraints, error-handling requirements, and test hints. The model was instructed to generate code satisfying all specified constraints and to append a brief (1-paragraph) rationale. The template schema is documented in the reproducibility package.

C Task Materials and Metrics

This section provides expanded static metrics for the three programming tasks described in Method Section 3.3. Table 12 reports structural complexity, test coverage, and mutation adequacy for starter code and reference implementations.

These metrics confirm the intended structural ordering (Task A: low-to-moderate; Task B: moderate; Task C: moderate-to-high complexity) and adequate test sensitivity (mutation scores $\geq 60\%$).

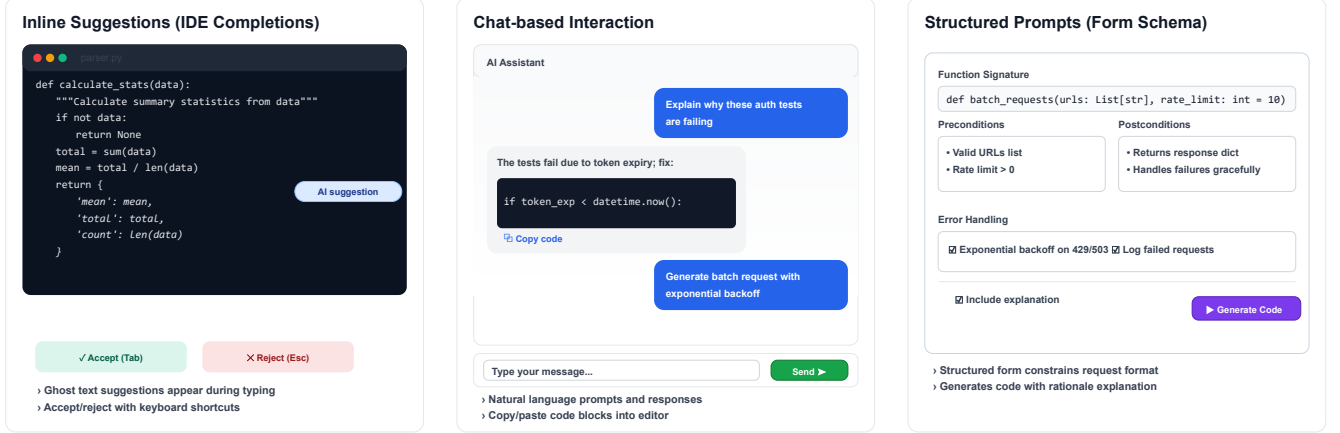


Figure 7: Interaction modes used in the study. Left—Inline Suggestions: ghost-text completions in the editor with keyboard accept/reject. Middle—Chat-based Interaction: prompt/response panel with copyable code blocks. Right—Structured Prompts: a form schema (function signature, pre/post-conditions, error handling) that generates code and a brief rationale. Visuals are representative UI snapshots; only the UI differed, the LLM backend and decoding settings were identical.

Table 12: Expanded static metrics and test adequacy for Tasks A–C. Metrics computed on starter code; coverage and mutation scores computed on reference implementations.

Metric	Task A	Task B	Task C
Total LOC (starter)	203	267	314
Median cyclomatic	3.9	4.8	6.1
Max cyclomatic	8	11	14
Avg fan-in	1.2	1.8	2.3
Avg fan-out	2.1	2.7	3.4
Line coverage (%)	84	82	81
Branch coverage (%)	78	76	77
Mutation score (%)	64	62	61

D Study Design, Preregistration, and Power

D.1 Preregistration and Deviations

The study was preregistered before data collection commenced. The following deviations occurred during execution:

- (1) We added the mode-agnostic verification-load composite V_{core} and its extended V_{plus} as mechanistic variables. These were not explicit in preregistration but derive from pre-registered log variables (compile/test failures, time-to-first-compile, churn, pauses, context switches).
- (2) We report Johnson–Neyman regions for continuous Complexity (exploratory moderator) and Inline acceptance (exploratory behavioral moderator) to identify complexity-dependent crossover points.
- (3) We include ordinal cumulative-link mixed models (CLMM) as robustness checks for TLX and accelerated failure-time (AFT) models for time to address right-censoring.

All deviations were analytic refinements that did not alter the core design or primary outcomes. Primary inferences reported in Results Section 5 remain consistent with preregistered hypotheses.

D.2 Power and Sample Size

We targeted $N=60$ (AI cohort $n=36$, Control cohort $n=24$) based on simulation-based power analysis using pilot intraclass correlations. Table 13 summarizes the assumptions and detectable effects.

Table 13: Power assumptions and detectable effects (simulated). Simulations used 1,000 draws with pilot ICC estimates from a 6-participant pilot study.

Outcome	ICC (pilot)	Effect metric	80% power detectability
RAW–TLX	0.22	Mode main effect f	≈ 0.18 (small-to-medium)
Log-time	0.24	Mode main effect f	≈ 0.18 (small-to-medium)
AI vs Control	–	Between-group δ	≈ 0.45 (medium)

With the achieved sample size, we had adequate power to detect small-to-medium within-subjects Mode effects on workload and time, and medium AI vs. Control differences, at $\alpha=0.05$ with Holm correction. Simulation code and sensitivity analyses (varying ICC assumptions) are available in the reproducibility package.

D.3 Randomization and Counterbalancing

This subsection details the decoupled Task×Mode schedule (Method Section 3.3) and allocation procedures.

Sequence generation. We enumerated 9 perfect matchings $\{S1 \dots S9\}$ between Tasks $\{A,B,C\}$ and Modes $\{Inline, Chat, Structured\}$, rotated task positions 1–3 to balance Index effects. Each sequence was instantiated exactly 4 times, yielding 36 AI participants. The Control cohort used all 6 task-order permutations (no Mode), with 4 participants per permutation (24 total).

Blocked randomization. Assignment to sequences was blocked by ExperienceBin (novice/intermediate/experienced) and AI_Familiarity (weekly-daily vs. monthly-rarely) to ensure balanced distribution across conditions. Allocation was concealed until session scheduling.

Balance checks. By construction, allocation was uniform within each sequence (4 participants per sequence). Fisher’s exact tests confirmed no association between assigned sequence and Experience-Bin (AI cohort: $p=0.88$; Control cohort: $p=0.90$). No dropouts or protocol violations occurred; all 60 participants completed the full study.

E Measurement Instruments (Full Details)

This section provides comprehensive measurement specifications referenced in Method Section 3.5. Table 15 summarizes all constructs, instruments, scales, timing, data sources, and analytic notes. We then detail scoring procedures, validation evidence, and psychometric properties for each instrument.

Table 14: Unified interaction/event log schema (all timestamps are monotonic, synchronized to IDE clock).

Field	Description
participant_id,	Identifiers and condition
task_id, mode	
ts, event_type	Timestamp; {offer, accept, reject, edit, prompt, response, paste, compile, test_run}
payload	Event-specific JSON (e.g., prompt text hash, response tokens, lines changed)
cursor_loc, file_path	Source location metadata
compile_status, test_summary	Build/run status; tests passed/failed

E.1 Subjective Instruments

RAW-TLX (composite of 4 subscales). We administered Mental Demand, Temporal Demand, Effort, and Frustration on 0–100 VAS sliders with standard left/right anchors. Composite = mean of subscales. Internal consistency in-study: $\alpha=0.86$.

SSSQ (Short Stress State Questionnaire). Standard scoring into Worry, Task Engagement (reverse), and Distress; we analyzed the total z-score post-task and Δ (post–pre). Internal consistency: $\alpha=0.81$.

KSS (Karolinska Sleepiness Scale). 9-point single item: 1 (Extremely alert) → 9 (Very sleepy, great effort to keep awake). Analyzed post-task and Δ . Test–retest at Task 1 baselines: $r=0.72$.

UMUX-Lite (usability). Two 7-point items; standardized to 0–100.

Trust in Automation (short form). Four 7-point items; composite mean.

E.2 Behavioral Indicators and Verification-Load Computation

This subsection documents the computation of behavioral indicators, including the verification-load composites (V_{core} and V_{plus}).

Inline acceptance rate (definition). Acceptance rate is computed at the task level as:

$$\text{acceptance} = \frac{\text{accept} + \text{edited-after-accept}}{\text{accept} + \text{edited-after-accept} + \text{reject}}.$$

Partial-accept events that the IDE logs as accept are counted as accepts. Edits following an accept (edited-after-accept) remain counted as accepts to reflect real-world acceptance with subsequent refinement. Rejections (reject) are counted in the denominator; offers without an explicit accept/reject decision are excluded from the computation.

Components of V_{core} . The mode-agnostic verification-load composite aggregates five signals available in all interaction modes:

- (1) Number of compile/test failures prior to first full pass
- (2) Time-to-first-compile (seconds from task start to first compile attempt)
- (3) Normalized code churn: (insertions + deletions) / total file size
- (4) Cumulative pause time: sum of pauses >2 s (seconds)
- (5) Context switches: editor $\leftrightarrow$ tests $\leftrightarrow$ chat/form

Standardization procedure. Each component is z-standardized within task across all participants to remove task-scale differences. For participant i on task t :

$$V_{core,it} = \frac{1}{5} \sum_{k=1}^5 z_t(X_{it}^{(k)}),$$

where $z_t(\cdot)$ denotes within-task standardization.

Extended index V_{plus} . For sensitivity analyses, we constructed an extended index that adds mode-specific signals: (vi) Inline rejection density (rejections per minute) and (vii) Chat prompt turns per task. Each mode-specific signal is z-standardized within mode to ensure commensurability. V_{plus} was used for robustness checks and exploratory decompositions; primary mediation analyses (Results, Section 5.3) use V_{core} to preserve mode-agnostic interpretation.

Reliability and internal structure. Cronbach’s α for the five V_{core} components was 0.80, indicating acceptable internal consistency. Principal components analysis yielded a dominant first factor (eigenvalue = 2.61, 52.2% variance explained) with high loadings on failures (0.78), time-to-first-compile (0.72), and pauses (0.68), moderate loading on switches (0.63), and lower loading on churn (0.49). Table 16 reports inter-component correlations.

F Data Processing, Quality Control, and Missingness

This section documents data preprocessing, quality control procedures, and missingness handling referenced in Method Section 4. These procedures ensure data integrity and support the validity of statistical inferences.

Clock synchronization. IDE and test-harness timestamps were aligned via a 10 s handshake protocol at task start. Residual drift (mean <50 ms) was verified via compile/test event alignment and corrected during log harmonization. This ensures accurate temporal sequencing of verification-load components.

Outlier handling. Continuous outcomes were inspected via boxplots and quantile-quantile plots. Values exceeding $3 \times \text{IQR}$ beyond the first/third quartiles were flagged; winsorization was applied when visual inspection of residuals indicated leverage effects. Time outcomes were modeled on the log scale to address positive skew; tasks exceeding the 40 min cap were right-censored and analyzed

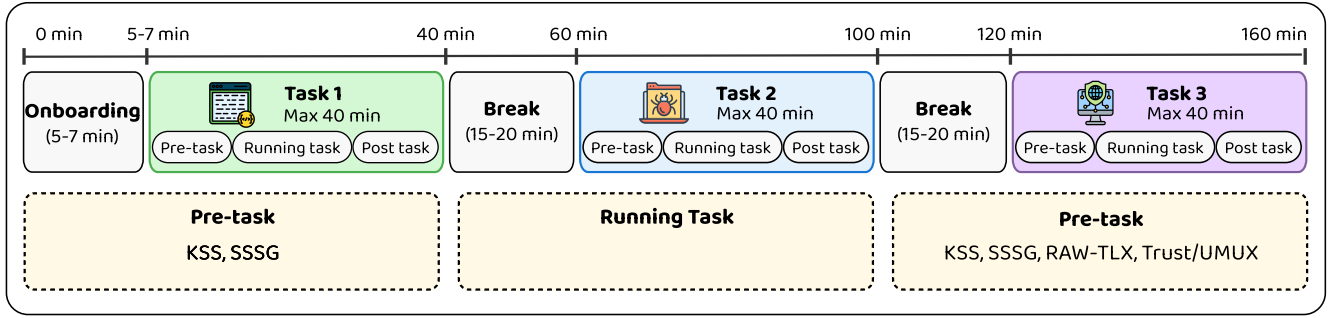


Figure 8: Within-task measurement timeline and between-task breaks. Each task block includes: pre-task measures (KSS, SSSQ), a task period instrumented with IDE/build/test logging, and post-task measures (RAW TLX, KSS, SSSQ). Logged signals feeding the verification-load composite include compile/test failures, time-to-first-compile, normalized code churn, cumulative pauses (>2 s), and context switches (editor $\leftrightarrow$ tests $\leftrightarrow$ chat/form). Blocks are separated by 15–20 min breaks; a brief next-day follow-up repeats the same timeline.

Table 15: Comprehensive measurement overview by construct, instrument/metric, scale, timing, data source, and analytic notes. VAS: visual analog scale. EMM: estimated marginal mean.

Category	Construct	Instrument / Metric	Scale	Timing	Source	Notes / Indices
Subjective	Workload	RAW NASA-TLX subscales: Mental, Temporal, Effort, Frustration; composite (mean)	0–100 (VAS)	Post-task	Self-report	Report subscales & composite; α target $\geq .80$
Subjective	Stress	Short Stress State Questionnaire (SSSQ)	per SSSQ	Pre & Post	Self-report	Analyze post-task and Δ (post-pre)
Subjective	Fatigue	Karolinska Sleepiness Scale (KSS)	KSS 1–9	Pre & Post	Self-report	Analyze post-task and Δ (post-pre)
Subjective	Trust / Usability	Trust in Automation (short form); UMUX-Lite (usability)	1–7 Likert	Post-task	Self-report	Mode-level face validity; exploratory covariate
Objective	Suggestion dynamics (Inline)	Acceptance rate = $\frac{\text{accept} + \text{edited-after-accept}}{\text{accept} + \text{edited-after-accept} + \text{reject}}$; rejection density (per min)	Fraction count	During	IDE logs	Used in V_{plus} (secondary). Acceptance definition matches Appendix E.2.
Objective	Prompting effort (Chat)	Prompt turns; code-block extractions per turn	Counts	During	Chat logs	Used in V_{plus} (secondary)
Objective	Code churn	Lines inserted / deleted (normalized by file size)	Counts (norm.)	During	IDE/VCS diffs	Contributes to verification-load composite
Objective	Build/test friction	Compile/test failures; time-to-first-compile; test-run count	Counts / time	During	Build & test logs	Primary V_{core} signals: failures, time-to-first-compile; test-run count exploratory
Objective	Attention / flow	Pause durations >2 s; context switches (editor $\leftrightarrow$ test $\leftrightarrow$ chat/form)	Time counts	During	IDE activity logs	Exploratory workload proxies
Objective	Verification-load (V_{core})	Z-scored mean of mode-agnostic signals: compile/test failures, time-to-first-compile, normalized code churn, cumulative pauses, context switches	Unitless (z)	Derived	Aggregated logs	Primary mechanistic index; reliability (α) and PCA sensitivity reported; used in mediation
Objective	Verification-load (V_{plus})	V_{core} + mode-specific signals: Inline acceptance/rejection; Chat prompt turns	Unitless (z)	Derived	Aggregated logs	Secondary/robustness analyses; not used in primary mediation
Performance	Completion time	Minutes from first keystroke to first full pass	Minutes	Post-task	Timer / logs	log-transform if skewed; EMMs with 95% CIs
Performance	Correctness	Tests passed out of N (per task $N = 10-12$)	Binomial	Post-task	Test harness	GLMM (logit) with task-level trials
Performance	Mutation survivors (opt.)	Surviving mutants in reference suite	Count	Post-task	Mutmut	Robustness check for oracle sensitivity

with AFT models as sensitivity checks (Results confirm LMM inferences).

Missingness analysis. Overall missingness across all measures was $< 1.7\%$. Little’s MCAR test indicated missingness was completely at random ($p=0.41$). Primary analyses used listwise deletion;

Table 16: V_{core} component correlations (pooled across tasks; Pearson r). Positive correlations indicate that verification frictions tend to co-occur.

	Failures	TTF-compile	Pauses	Switches	Churn
Failures	1.00	0.41	0.29	0.25	0.18
TTF-compile		1.00	0.33	0.22	0.21
Pauses			1.00	0.27	0.19
Switches				1.00	0.16
Churn					1.00

sensitivity analyses with multiple imputation by chained equations (MICE; 10 imputations) reproduced the same pattern of results for all primary outcomes. Table 17 summarizes missingness by variable family.

Table 17: Missingness by variable family. All missingness rates were below 2% and treated as MCAR.

Family	Pct missing	Notes
RAW-TLX	0.6%	1 skipped slider
KSS/SSSQ	1.2%	2 post-task omissions
Event logs	0.4%	1 partial session; non-critical events recovered

G Statistical Model Specifications

This section provides complete model formulas and estimation details for analyses reported in Results Section 5. All models were estimated in R (version 4.3.1) using `lme4` for mixed-effects models, `lmerTest` for Satterthwaite degrees-of-freedom approximation, and `emmeans` for marginal means and contrasts.

G.1 Linear Mixed-Effects Models (LMMs)

Model family. Gaussian LMMs for continuous outcomes (RAW-TLX composite, $\log(\text{time})$, KSS, SSSQ) with Satterthwaite degrees-of-freedom approximation for Type-III tests and confidence intervals. Binomial GLMM (logit link) for correctness (trials = N_{tests} per task). Random intercepts for Participant and TaskVersion; random slopes for Index by Participant where convergent (assessed via AIC comparison).

Fixed effects structure. For RQ1 (mode effects within AI cohort):

$$Y \sim \text{Mode} \times \text{Task} + \text{Index} + \text{ExperienceBin} + \text{AI_Familiarity} + \text{PreTaskComplexity} + \text{Environment}$$

For RQ2 (AI vs. Control trajectories):

$$Y \sim \text{Condition} \times \text{Index} + \text{Task} + \text{ExperienceBin} + \text{AI_Familiarity} + \text{PreTaskComplexity} + \text{Environment}$$

Random effects: $(1 + \text{Index} \mid \text{Participant}) + (1 \mid \text{TaskVersion})$.

G.2 Contrasts and Multiplicity Control

Pairwise contrasts. Estimated marginal means (EMMs) and pairwise contrasts computed via `emmeans` package. Holm adjustments applied within pre-specified contrast families: (i) Mode-by-Task

cells (9 comparisons per outcome); (ii) Condition $\times$ Index trajectories (2 slopes); (iii) moderator strata (Experience/Complexity subgroups).

G.3 Mediation Analyses

Two-equation approach. Following Hayes [14], we estimated:

$$V_{\text{core}} \sim \text{Mode} \times \text{Task} + \text{Index} + \dots \quad (a\text{-path})$$

$$Y \sim V_{\text{core}} + \text{Mode} \times \text{Task} + \text{Index} + \dots \quad (b\text{-path})$$

Indirect effects (ab) estimated via Monte Carlo simulation (10,000 draws) with cluster-robust standard errors. Bias-corrected bootstrapped 95% confidence intervals (2,000 resamples) used as sensitivity checks. We interpret significant indirect effects as consistent with *partial mediation* rather than causal identification, given the observational nature of V_{core} .

G.4 Equivalence Testing

TOST procedure. For non-significant pairwise differences of practical interest, we conducted two one-sided tests (TOST) to assess equivalence within pre-specified bounds: ± 5 TLX points (on 0–100 scale), $\pm 5\%$ on log-time scale (time ratios 0.95–1.05), and ± 0.5 tests passed (on 10-test scale). Bounds reflect minimally important differences from pilot data and prior literature.

H Data Analysis Details

H.1 Preprocessing, Exclusions, and Variable Construction

- Log harmonization, timestamp alignment, event de-duplication.
- Outlier policy (winsorization), transformations (log-time), and z-scaling.
- Construction of V_{core} (components, standardization within task, reliability/PCA).
- Pre-registered exclusion rules and counts.

H.2 Model Specifications

Mode effects (AI cohort).

$$Y \sim \text{Mode} \times \text{Task} + \text{Index} + \text{ExperienceBin} + \text{AI_Familiarity} + \text{PreTaskComplexity} + \text{Environment} + (1 + \text{Index} \mid \text{Participant}) + (1 \mid \text{TaskVersion})$$

Condition trajectories (AI vs. Control).

$$Y \sim \text{Condition} \times \text{Index} + \text{Task} + \text{ExperienceBin} + \text{AI_Familiarity} + \text{PreTaskComplexity} + \text{Environment} + (1 + \text{Index} \mid \text{Participant}) + (1 \mid \text{TaskVersion})$$

Correctness (binomial GLMM).

$$\text{Passed} \sim \text{Mode} \times \text{Task} + \dots + (1 \mid \text{Participant}) + (1 \mid \text{TaskVersion}), \quad \text{trials} = N_{\text{tests}}$$

Time outcomes. LMM on $\log(\text{minutes})$; sensitivity via AFT (log-normal).

H.3 Mediation by Verification-Load

$$V_{\text{core}} \sim \text{Mode} \times \text{Task} + \text{Index} + \dots$$

$$Y \sim V_{\text{core}} + \text{Mode} \times \text{Task} + \text{Index} + \dots$$

Indirect effects via Monte Carlo (10,000 draws) with cluster-robust bootstraps; report a , b , ab , CIs, and proportion mediated.

H.4 Moderation and Johnson–Neyman Procedure

- Mode×ExperienceBin (EMMs by stratum).
- Mode×Complexity (continuous); J–N crossover points with CIs.

H.5 Multiplicity, Equivalence, and Carryover

Holm adjustments by contrast family; TOST settings and margins; lagged Mode carryover checks.

H.6 Diagnostics and Robustness

Residual plots, influence checks, random-effects structures, alternative links (Poisson/NegBin for correctness), censoring sensitivity (AFT), and multiple-imputation sensitivity if applicable.

H.7 Qualitative Methods and Integration

Protocol, coding process, reliability statistics, theme definitions, and joint display construction tying themes to V_{core} components.

H.8 Power and Sensitivity Calculations

Design parameters, simulation code summary, and detectable effect sizes for within- and between-subjects contrasts.

H.9 Reproducibility Package

Pointers to scripts for data preparation, modeling, figure generation, and the anonymized artifact bundle.

I Additional Results Tables

This section provides extended statistical results referenced in Results Sections 5.2–5.4, including Type-III tests, pairwise contrasts, per-mode trajectory slopes, and exploratory follow-up analyses.

I.1 Type-III Likelihood-Ratio Tests

Table 18 reports omnibus Type-III likelihood-ratio tests for primary models. These tests assess the overall significance of main effects and interactions before pairwise contrasts.

Table 18: Type-III LRTs for primary models (AI cohort for Mode effects; all participants for Condition trajectories). Significance codes: * $p < 0.05$, * $p < 0.001$.**

Effect	TLX χ^2 (df)	Log-time χ^2 (df)	Correctness χ^2 (df)
Mode	30.7 (2)***	27.6 (2)***	2.9 (2)
Mode×Task	12.9 (4)*	11.1 (4)*	12.1 (4)*
Condition	42.3 (1)***	36.8 (1)***	18.2 (1)***
Condition×Index	2.1 (1)	1.8 (1)	0.7 (1)

I.2 Mode-by-Task Pairwise Contrasts

Table 19 reports selected pairwise contrasts within Mode×Task cells, with Holm-adjusted p -values. Negative TLX/time differences favor the first (row) mode.

Table 19: Selected Mode contrasts by Task (Holm-adjusted p). Negative TLX/time differences favor the row Mode.

Contrast	TLX Δ	Time ratio	Correctness $\Delta/10$
A: Inline vs Chat	−6.7 ($p=0.006$)	0.81 ($p < 0.001$)	+0.1 ($p=0.56$)
A: Inline vs Str.	−8.8 ($p < 0.001$)	0.79 ($p < 0.001$)	0.0 ($p=0.93$)
B: Inline vs Str.	−5.5 ($p=0.024$)	0.90 ($p=0.014$)	−0.4 ($p=0.041$)
C: Chat vs Inline	−1.2 ($p=0.48$)	0.97 ($p=0.56$)	+0.7 ($p=0.009$)
C: Chat vs Str.	−6.3 ($p=0.008$)	0.91 ($p=0.021$)	+0.9 ($p=0.005$)

I.3 AI vs Control Aggregate Effects

Table 20 reports pooled AI vs. Control comparisons across all tasks.

Table 20: AI vs Control aggregate effects (EMM differences/ratios; 95% CI). All comparisons favor AI.

	TLX (pts)	Time ratio	Correctness OR
AI vs Control	−18.2 [−22.3, −14.0]	0.78 [0.74, 0.83]	1.71 [1.37, 2.13]

I.4 Per-Mode Fatigue Trajectories

Table 21 reports fatigue (KSS) slopes across Task Index for each mode within the AI cohort, as referenced in Results Section 5.3.

Table 21: Per-mode fatigue (KSS) slope across Task Index (AI cohort only): adjusted change from Task 1 to Task 3 (Δ). SSSQ showed the same ordering but did not reach significance after correction.

Mode	KSS Δ	Note
Inline	+1.18	Steeper than Chat ($p=0.048$)
Structured	+1.05	Intermediate
Chat	+0.92	Shallowest

I.5 Exploratory Next-Day Follow-Up

Table 22 reports adjusted means (EMMs) with 95% CIs for the exploratory next-day follow-up by Condition (AI vs. Control), as described in Method Section 3.1. Models mirror the main analysis (Condition, Task, and environment covariates; participant random intercept), but we treat these estimates as descriptive given the smaller sample and limited power. Patterns are consistent with the main session: AI remains faster on average (by ~21–23%) and achieves higher correctness, while post-task fatigue (KSS) and stress (SSSQ z) are comparable between groups within wide CIs. No estimate materially contradicts the primary conclusions.

Table 22: Exploratory next-day follow-up: adjusted means (EMMs) with 95% CIs. Descriptive models parallel main-session specifications; we refrain from multiple-comparison testing.

Condition	KSS (post)	SSSQ (post, z)	Time (min)	Correctness (/10)
AI	3.5 [3.1, 3.9]	0.08 [-0.08, 0.23]	24.6 [22.8, 26.6]	7.7 [7.3, 8.1]
Control	3.4 [3.0, 3.8]	0.05 [-0.11, 0.21]	31.1 [28.5, 34.0]	6.5 [6.0, 7.0]

Notes: (i) EMMs are adjusted for Task and environment with participant random intercepts; (ii) CIs reflect the smaller follow-up subset and are therefore wider than in the main session; (iii) Estimates are descriptive and not used in primary inference. Patterns align with the main results: AI remains faster and more correct on average, while post-task KSS/SSSQ differences are small and imprecisely estimated.

I.6 Robustness and Sensitivity Analyses

This subsection documents robustness checks for primary results, including alternative model specifications, influence diagnostics, multiple-imputation sensitivity, and composite-stability tests.

Alternative model specifications. Correctness results were consistent under (i) logistic GLMM (primary), (ii) Poisson and negative binomial count models (AIC within 2 of the primary), and (iii) linear probability models with cluster-robust SEs (all same-sign, comparable magnitudes). For time, accelerated failure time (AFT, log-normal) models accounting for right-censoring at 40 min reproduced LMM contrasts (Inline<Structured on Task A; Chat≈Inline on Task C; all $p < 0.05$ where reported). Ordinal cumulative-link mixed models (logit link) with participant/task-version random effects reproduced LMM inferences for TLX (same-sign Mode and Mode×Task effects).

Residuals and influence diagnostics. Quantile-quantile plots and residual-vs.-fitted profiles indicated acceptable normality and homoscedasticity after log-transforming time. Cook’s distance flagged 3/180 task-level observations as potential influences (top 1.7%); excluding them changed EMMs by < 0.6 TLX points and < 0.4 minutes. Leverage-based reweighting and Huber-robust fits yielded identical direction and significance of key contrasts.

Multiple imputation sensitivity. Results were unchanged under multiple imputation by chained equations (MICE; 10 imputations). All primary effects (Mode, Condition, interactions) retained significance and similar magnitudes. Pooled estimates fell within the confidence intervals of complete-case analyses.

Composite stability and alternatives. V_{core} showed acceptable internal consistency ($\alpha=0.80$). Equal-weighting (primary) and PCA-based weighting yielded qualitatively identical inferences. Using V_{plus} (adding Inline rejection density and Chat prompt turns) produced slightly larger indirect effects in mediation (Section 5.3) without reversing any conclusions.

Covariate and random-effects sensitivity. Including Environment (lab vs. remote) and AI_Familiarity did not alter the direction or significance of primary contrasts. Random slopes for Index by Participant improved fit where convergent (AIC ↓) with fixed-effect estimates materially unchanged.

Equivalence testing. Inline vs Chat on TLX in Task B was equivalent within ± 5 points ($p_{\text{TOST}}=0.024$), supporting the claim of no practically important difference at moderate complexity for intermediate users.

Carryover checks. Lagged Mode term (previous task’s mode as predictor) showed no reliable carryover effects (all $p > 0.10$), validating the counterbalanced design.

Summary of robustness. Across alternative model families, influence diagnostics, imputation methods, and data treatments, the pattern of findings remained stable: mode effects moderated by complexity and expertise, rising repeated-use burden, and partial mediation by verification-load (V_{core}).

J Qualitative Methods: Codebook and Reliability

This section documents the qualitative analysis procedures for post-task interviews, as described in Method Section 4 and reported in Results Section 5.7. We provide the coding protocol, inter-rater reliability statistics, and the final codebook.

Sampling and saturation. We coded all 60 post-task interviews (15–20 min each; total ~18 hours of audio). Thematic saturation was assessed after 48 interviews; the final 12 confirmed no new major themes emerged, indicating adequate coverage of the conceptual space.

Coding procedure. Two researchers independently open-coded a stratified 20% subset (12 interviews, stratified by Mode×Complexity) to develop a shared codebook. Cohen’s κ for major codes (speed vs. certainty, trust shocks, scaffolding, review fatigue) exceeded 0.70 before proceeding to full coding. Disagreements were resolved via discussion and documented in an audit trail (available in reproducibility package). Full coding was conducted in NVivo (version 14) with regular consistency checks.

Integration with quantitative results. We constructed joint displays (mixed-methods integration matrices) that map qualitative themes to (i) Mode×Complexity cells with highest prevalence, (ii) behavioral signatures from V_{core} components, and (iii) research questions (RQ1–RQ3). This approach grounds qualitative insights in quantitative patterns and vice versa.

Table 23: Abbreviated codebook used for thematic analysis. Full codebook with sub-codes and examples available in reproducibility package.

Theme	Definition (inclusion/exclusion)
Speed vs certainty	Preference for Inline on simple flow vs Chat for complex reasoning; include explicit mentions of "plan/steps"; exclude general satisfaction.
Trust shocks	Loss of trust after overconfident errors; include "looked right but failed tests"; exclude generic "LLM is wrong sometimes."
Scaffold me	Value in externalizing requirements via Structured; include mentions of pre/post-conditions; exclude general "forms are annoying."
Review fatigue	Rejection spirals prompting switching; include "after N rejections I switched"; exclude single rejection.

Table 23 presents the abbreviated codebook used for thematic analysis, including definitions and inclusion/exclusion criteria for each theme.